

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

Язык сценариев Javascript и библиотека jQuery

Урок № 8

Введение в jQuery

Содержание

Использование jQuery	4
Что такое jQuery	4
Подключение jQuery	7
Версии jQuery	10
Доступ к элементам страницы при помощи функции \$	11
Понятие селектора	14
CSS селекторы	14
jQuery селекторы	16
Селекторы форм	16
Особенности селекторов в jQuery	17
Методы управления стилями jQuery	18
Traversing. Методы обхода DOM	20
Обработка событий в jQuery	22
Назначение одного обработчика для нескольких событий	28
Анимационные методы jQuery	29

Методы, изменяющие высоту/ширину элемента и его видимость (display).....	30
Методы, изменяющие непрозрачность элемента (свойство opacity).....	33
Метод animate() для создания произвольных эффектов.....	35
Делегирование событий.....	37
Отмена обработчиков событий.....	43
Обработка данных форм.....	46
jQuery-плагины	51
jQuery и AJAX	58
Обработка ошибок при AJAX-запросе.....	68
Домашнее задание.....	71
Задание 1	71
Задание 2	72
Задание 3	73
Приложение	75

Материалы урока прикреплены к данному PDF-файлу. Для доступа к материалам, урок необходимо открыть в программе [Adobe Acrobat Reader](#).

Использование jQuery

Что такое jQuery

jQuery — это библиотека функций, написанная на JavaScript, позволяющая простыми методами манипулировать DOM-элементами, обрабатывать события, использовать анимацию, а также загружать или получать данные на основе AJAX.



Рисунок 1

Разработчиком jQuery является Джон Резиг (*John Resig*), который опубликовал первую версию этой библиотеки в 2006 году на компьютерной конференции «BarCamp» в Нью-Йорке. Она мгновенно стала популярной, т. к. решала несколько проблем, существенных для программистов на JavaScript:

- Позволяла легко обращаться к элементам, используя css-селекторы и различные фильтры.

- Была кроссбраузерной, т.е. работала одинаково хорошо и в Opera, и Mozilla Firefox, и в Internet Explorer, который был достаточно популярен в то время.
- Давала возможно делать анимационные эффекты в то время, когда css-свойство `transition`, `animation` и `@keyframes` еще не существовали.

В свое время Джон Резиг написал: «jQuery — это Javascript-библиотека, в основе которой лежит девиз: Программировать на Javascript должно быть приятно. jQuery берёт частые, повторяющиеся задачи, извлекает всю ненужную разметку и делает их короткими, элегантными и понятными».

В первой версии jQuery еще не имела средств работы с AJAX, но они были очень скоро добавлены. В январе 2006 года был создан первый плагин для работы с JSON-данными. Он показал, что jQuery может расширять функциональность за счет плагинов. Поэтому в скором времени таких плагинов было создано множество, что стало одной из причин успеха jQuery на долгие годы.

Сейчас разработка jQuery ведётся командой добровольцев на пожертвования. Об истории версий jQuery вы можете прочитать на [официальном сайте](#).

На данный момент популярность jQuery идет вниз, т. к. современные стандарты уже поддерживают многое из того, что изначально принесло популярность этой библиотеке:

- Выбор элементов по css-селектору (методы `document.querySelector()` и `document.querySelectorAll()`);
- Переключения классов (`element.classList.add()`, `element.classList.remove()`, `element.classList.toggle()`);

- Альтернативные способы передачи данных (Fetch API, а не AJAX).

Очень вероятно, что к появлению этих возможностей в стандарте JavaScript подтолкнула именно библиотека jQuery.

Зачем вам учить jQuery, если вы знаете JavaScript? Во-первых, затем, что она упрощает понимание кода и позволяет в 3 строчки сделать то, что на нативном JS займет 10-15 строк, а иногда и больше. Во-вторых, затем, что на основе jQuery написано огромное количество плагинов, позволяющих решить какие угодно задачи, задействовав минимум кода и усилий. В-третьих, в сети существует масса сайтов, которые используют jQuery, и в случае добавления/изменения кода вам придется разбираться, что и как в них работает. В-четвертых, большинство современных CMS (Wordpress, Drupal, Joomla и другие) используют функциональность jQuery и для работы основных своих функций (меню, перетаскиваемые элементы, отправка AJAX-запросов и т. д.), так и для работы шаблонов (тем), плагинов и других расширений для этих CMS. В-пятых, “порог вхождения” в jQuery очень низкий, т. е. для того, чтобы разобраться, как и что работает в этой библиотеке, необходимо от нескольких часов до 2-3-х дней, даже, если вам никто об этом не рассказывает. Вряд ли вы столь же быстро разберетесь с React, Angular или Vue.js.

При этом вы должны понимать, что jQuery — это библиотека, написанная на JavaScript, то есть это не какая-то надстройка над JS, а переработанный и упрощенный подход к написанию кода, к тому же кроссбраузерный.

Подключение jQuery

Официальный сайт вы найдете по адресу <https://jquery.com/>. На нем всегда можно скачать свежую версию jQuery. На данный момент — это версия 3.5.1.

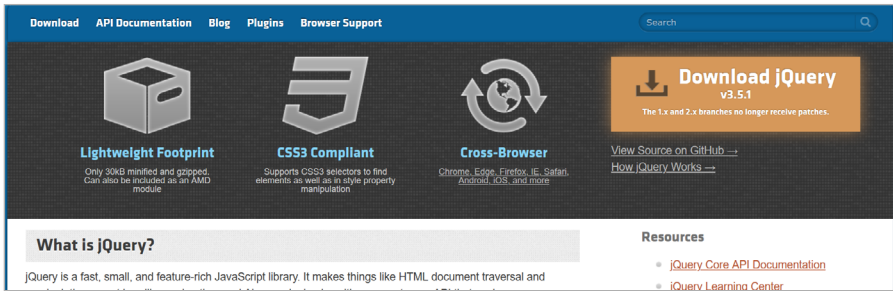


Рисунок 2

При клике на ссылку «Download jQuery» вы перейдете на страницу, которая позволит вам скачать последнюю версию библиотеки в нескольких вариантах:

- **Compressed, production jQuery** — сжатый, минифицированный файл, который стоит использовать для работы со страницами сайта. То есть весит этот файл вдвое меньше, чем несжатый, что ускоряет загрузку сайта, но он практически нечитабелен, а значит, код в нем вам будет непонятен.
- **Uncompressed, development jQuery** — несжатый файл для разработчиков плагинов с комментариями ко многим функциям. Это файл с читабельным кодом позволяет понять, как устроена jQuery изнутри и воспользоваться ее функциями для написания расширения.
- **Slim build** — «тонкая сборка», т. е. уменьшенная версия jQuery, откуда убраны все анимационные методы

и методы для работы с AJAX. Соответственно, использовать в коде вы их не сможете. Это вариант для тех, кто не собирается ничего загружать или отправлять с помощью AJAX-технологии, а вместо анимационных методов будет использовать CSS-свойства `@keyframes`, `animation` и `transition`.

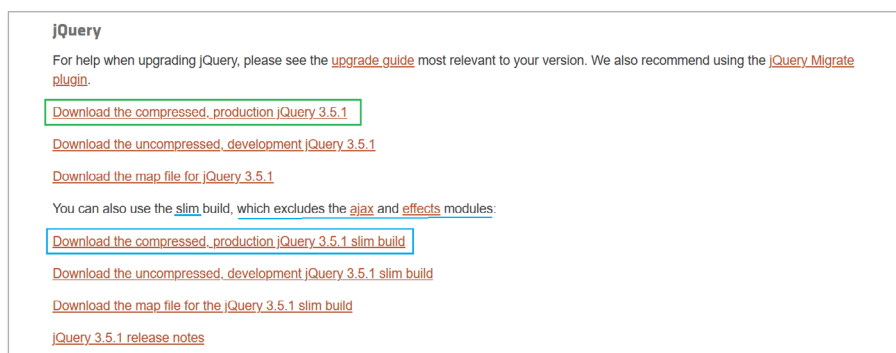


Рисунок 3

Мы будем использовать для разбора HTML-страницы с примером compressed-версию jQuery. Именно ее и стоит скачать. Учтите, что на момент чтения вами этого урока номер версии jQuery может отличаться. Имеет смысл скачать именно последнюю версию, т.к. она является наиболее актуальной.

Для того чтобы загрузить выбранную версию jQuery кликните на ссылке, нажмите **CTRL+S** и сохраните файл в нужную вам папку со всеми файлами вашего сайта. Чаще всего ее сохраняют в папку `js`, в которой хранят и другие JS-файлы. Для подключения jQuery в файл необходимо указать в атрибуте `src` тега `<script>` путь к загруженному файлу с этой библиотекой, например, в блоке `<head>`:

```
<head>
  <meta charset="UTF-8">
  <title>Index</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/jquery.min.js"></script>
  <script src="js/your-script.js"></script>
</head>
```

... или перед закрывающимся `</body>`:

```
  <script src="js/jquery.min.js"></script>
  <script src="js/your-script.js"></script>
</body>
</html>
```

Обратите внимание на строку

```
<script src="js/your-script.js"></script>
```

Она идет следом за подключением jQuery и содержит ссылку на файл скрипта, в котором вы будете писать код, используя функциональность jQuery.

Помимо подключения jQuery из папки вашего проекта, довольно распространенным является способ подключения этой библиотеки с CDN — сетей распределенного контента, которые позволяют загружать множество популярных библиотек, плагинов и фреймворков, указав ссылку на них. Для загрузки jQuery вы можете использовать такие CDN:

- [jQuery CDN](#);
- [Google CDN](#);
- [Microsoft CDN](#);

- [CDNJS CDN](#);
- [jsDelivr CDN](#).

В этом случае подключение jQuery будет выглядеть так:

```
<script src="https://ajax.googleapis.com/ajax/libs/
  jquery/3.5.1/jquery.min.js"></script>
<script src="js/your-script.js"></script>
```

Плюсом такого подключения является то, что вы можете писать код, не имея на компьютере скачанной версии jQuery, и он будет работать. Также есть вероятность, что вы или пользователь, который будет просматривать в браузере вашу html-страницу (сайт), уже загружал jQuery с этого ресурса, и она осталась в кэше браузера. Тогда не придется тратить время на загрузку этой библиотеки.

К минусам загрузки jQuery с CDN можно отнести то, что при отключенном Интернете вы не сможете писать код на jQuery, т. к. не будет описания функций, которые вы наверняка используете в коде. В этом случае ваш скрипт выдаст ошибку вида

```
Uncaught ReferenceError: $ is not defined
```

Версии jQuery

На данный момент принято использовать jQuery версии 3*, т.е. 3.0, 3.1, 3.2 и т.д. Они основаны на стандартах ES6, ES7, поддерживают новые возможности типа промисов, цикла [for..of](#), нового API для работы с анимацией и [другие \(рус\)](#).

Для совместимости со всеми старыми браузерами стоит использовать версии линейки 1*, т. к. они поддер-

живают совместимость с браузером Internet Explorer v6-8. Самой последней из них является версия 1.12.4.

В версиях 2.* от поддержки старых IE отказались. Поэтому линейка версий 2.* немного легче из-за вырезанного кода с поддержкой IE v.6-8. Она является промежуточной между версиями 1 и 3, и встречается в основном на тех сайтах, где была подключена во время ее выпуска.

Доступ к элементам страницы при помощи функции \$

В jQuery к любому объекту необходимо обращаться, используя ключевое слово jQuery или его замену — \$. Для того, чтобы использовать эту библиотеку в заголовочной части страницы, когда весь контент еще не загрузился, необходимо записать весь код внутри такой функции:

```
jQuery(document).ready(function() {  
    //ваш код  
});  
  
$(document).ready(function() {  
    //ваш код  
});  
  
// используем стрелочную функцию  
$(document).ready(() => {  
    $('.block').addClass('active');  
    //ваш код  
});  
  
// укороченный вариант  
$(function() {  
    $('.nav-link').addClass('active');  
    //ваш код  
});
```

```
// используем стрелочную функцию
$( () => {
  $('<div>.block').attr('data-text', 'Test Block');
  //ваш код
});
```

Функций много, но на самом деле все они выполняют одно и то же — отслеживают момент загрузки DOM-дерева документа, чтобы позволить скрипту с ним работать без ошибок. Какую выбрать вы решаете сами в зависимости от того, насколько вы поняли синтаксис стрелочных функций или функций-коллбэков.

В большинстве руководств по jQuery вы найдете вариант

```
$(document).ready(function() {
  //ваш код
});
```

Можете использовать именно его.

В том случае, если вы подключаете свой скрипт перед закрывающимся тегом `</body>`, код в синтаксисе jQuery можно записывать без `$(document).ready()`.

Для того чтобы разобраться с возможностями jQuery, мы рассмотрим с вами, каким образом можно добавить интерактивности для страницы воображаемой онлайн-библиотеки, состоящей из шапки, нескольких разделов и подвала. Готовый пример с HTML-разметкой, CSS-стилями и кодом на jQuery вы найдете в папке *examples/books* этого урока. Файл *index.html* в этой папке предполагает подключение jQuery и плагинов с CDN, а *index-local.html* — из папки *books/js*.

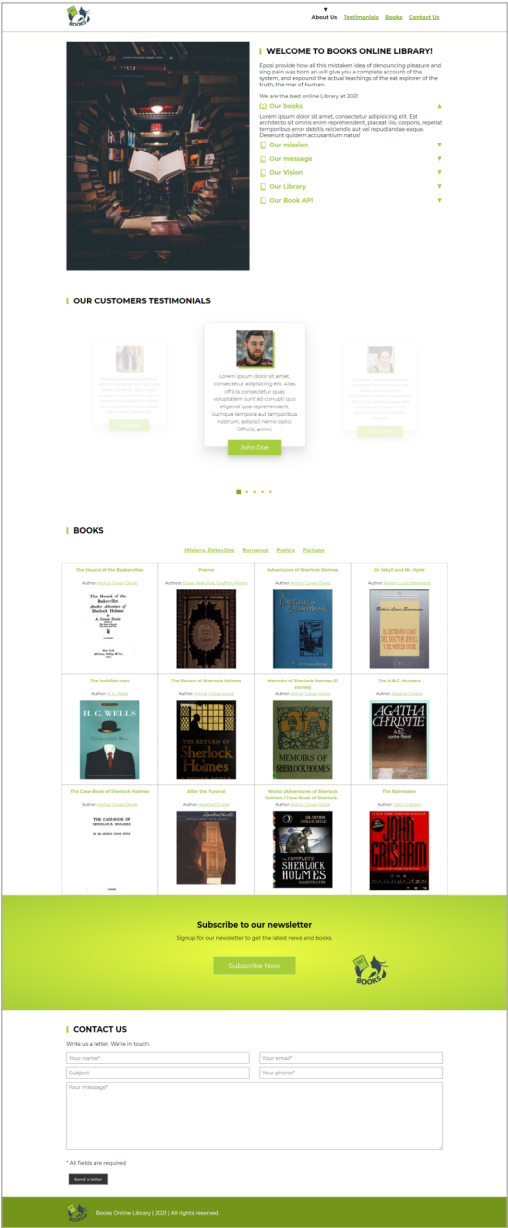


Рисунок 4

Поскольку на этой странице существует масса html-элементов, первое, с чем нам необходимо разобраться — это как выбирать из всего множества нужные элементы с помощью jQuery. Для этой цели мы будем использовать селекторы jQuery. Сначала познакомимся с теорией.

Понятие селектора

Понятие селектора должно быть вам знакомо из CSS и JavaScript, если вы использовали методы `document.querySelector()` или `document.querySelectorAll()`. В jQuery селектором является строка, в которой задаются условия поиска DOM-элементов на странице. В качестве такой строки можно использовать CSS-селекторы или специальные jQuery-селекторы, которые еще называют фильтрами.

CSS селекторы

Общий вид записи CSS-селектора выглядит так:

```
$ ( 'селектор' )
```

Например, `$('#date-now')` предоставит вам доступ к единственному элементу на странице, который имеет атрибут `id="date-now"`. Селектор `$(".accordion-header")` позволит обращаться сразу к нескольким элементам с классом `.accordion-header`, а селектор `$('[href^="#"]')` даст доступ ко всем ссылкам, у которых атрибут `href` начинается с `#`. Такой селектор, как `$(".block:nth-child(3n)")` позволит выбрать каждый 3-й элемент с классом `.block`. То есть выбор элементов осуществится так же, как это

работает с CSS-селекторами. Только в случае jQuery мы будем вызывать для этих селекторов различные методы jQuery и назначать обработчики событий, а не только задавать CSS-свойства.

CSS-селекторы в jQuery фактически опираются на правила записи селекторов в CSS, поэтому, если вы хорошо разбираетесь в CSS, то обращение к элементам для вас не будет представлять труда.

В таблице 1 приложения вы найдете различные CSS-селекторы, используемые в jQuery.

Практическое задание: Необходимо сделать так, чтобы год в подвале страницы всегда соответствовал текущему. В разметке год находится в пустом элементе `<span id="date-now"></span>`:

```
<p>Books Online Library | <span id="date-now"></span>  
| All rights reserved.</p>
```

Мы обратимся к этому элементу с помощью CSS-селектора и установим текст в нем, исходя из текущей даты и года, который можем получить с помощью метода `getFullYear()` объекта `Date`:

```
$('#date-now').text(new Date().getFullYear());
```

Метод `text("some text")` в этой строке дает нам возможность записать любой текст в указанном в селекторе элементе. Если вы используете метод `text()` без параметров, то получите тот текст, который в данный момент записан внутри элемента (аналог JS свойства `textContent`).

*Рисунок 5*

jQuery селекторы

jQuery-селекторы записываются так:

```
$(':селектор')
```

Например, `$("section:even").css("background", "gray")` позволит вам залить все четные элементы `<section>` серым цветом фона. Селектор `$(":header")` даст возможность управлять всеми заголовками на странице, вне зависимости от того, какой это тег — `h1`, `h2`, `h3` или `h4`. Селектор `$(".block:hidden")` позволит выбрать элементы с классом `.block`, которые были скрыты с помощью CSS или JavaScript (то есть имеют свойство `display: none`, заданное для класса или для отдельного элемента, или атрибут `hidden`, или `type=hidden`).

Как правило jQuery-селекторы позволяют найти либо дочерние элементы по определенному признаку, либо элементы с определенными характеристиками (скрытые, анимированные и т. п.), либо определенные элементы форм. Их варианты представлены в 2-х таблицах приложения.

Селекторы форм

Эти селекторы используются для управления элементами форм: полями ввода, переключателями, флажками, выпадающими списками и кнопками.

Предположим, что вам нужно сделать недоступной кнопку в форме, пока пользователь не согласится с условиями предоставления услуг. Для этого подойдет строка

```
$(':button').prop('disabled', true);
```

В случае, если нужно, наоборот, эту кнопку сделать доступной, сработает селектор

```
$('button:disabled').prop('disabled', false);
```

Если поле с паролем нужно выделить рамкой (в случае неправильного заполнения, например), подойдет такой селектор:

```
$(':password').css('border', '2px solid red');
```

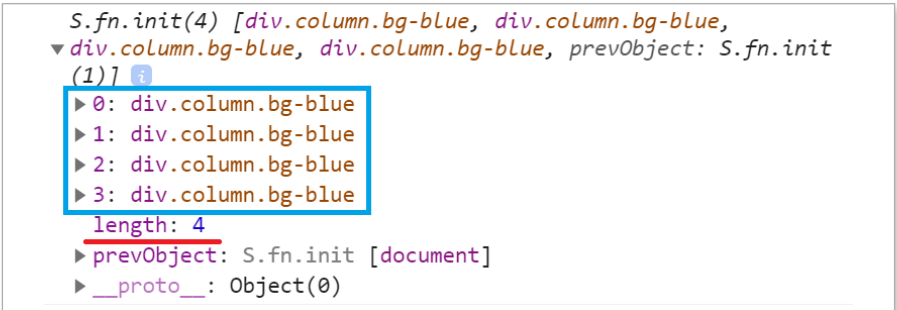
Множество других селекторов с примерами вы найдете в таблице 3 приложения.

Особенности селекторов в jQuery

Особенностью использования селекторов в jQuery является то, что функция jQuery, представленная обычно в виде \$ всегда возвращает объект, а не ошибку, если вы указали селектор неверно или таких селекторов на странице нет. Это имеет свои плюсы и минусы. К плюсам относится то, что код, написанный на jQuery выполнится и тогда, когда в обычном JavaScript будет ошибка типа «Cannot set property 'onclick' of null», и выполнение дальнейшего кода будет невозможно из-за нее. Минусом является то, что отследить, что вы сделали неверно, будет сложно, т.к. ошибок в консоли браузера нет, но и код тоже не работает.

Поэтому при использовании любого селектора вы можете проверить, есть ли такой объект, используя свойство `length`, которое всегда присутствует в объекте jQuery, и может быть выведено или отображено в консоли. Если `length` равен 0, то элемента, соответствующего вашему селектору, в DOM-дереве страницы нет.

Вид элементов, доступных на странице, выведенных в `console.log()`.

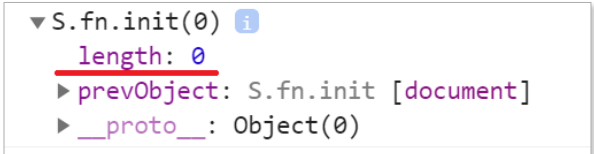


```

S.fn.init(4) [div.column.bg-blue, div.column.bg-blue,
▼ div.column.bg-blue, div.column.bg-blue, prevObject: S.fn.init
(1)] ⓘ
  ▶ 0: div.column.bg-blue
  ▶ 1: div.column.bg-blue
  ▶ 2: div.column.bg-blue
  ▶ 3: div.column.bg-blue
  length: 4
  ▶ prevObject: S.fn.init [document]
  ▶ __proto__: Object(0)
  
```

Рисунок 6

Вид элементов, не существующих на странице, выведенных в `console.log()`.



```

▼ S.fn.init(0) ⓘ
  length: 0
  ▶ prevObject: S.fn.init [document]
  ▶ __proto__: Object(0)
  
```

Рисунок 7

Методы управления стилями jQuery

Наверняка вы уже управляли стилями элементов на странице с помощью JavaScript. jQuery позволяет также

это сделать с помощью нескольких методов, которые либо добавляют атрибут `style` к указанному селектору (метод `css()`), или управляют добавлением/удалением атрибута `class` или значений внутри него.

Например, код `$('#block').css('color', 'red')` установит красный цвет текста для элемента с `id="block"` аналогично такой записи:

```
<div id="block" style="color: red">
  Lorem, ipsum dolor
</div>
```

Если необходимо задать несколько стилевых свойств, то синтаксис несколько меняется:

```
$('.example').css({
  color: 'red',
  'background-color': 'pink',
  fontSize: '18px'});
```

В этом случае вне зависимости от того, в каком формате — CSS или JS — были заданы свойства, они будут записаны в виде атрибута `style` сразу в нескольких элементах с классом `example`.

```
<div class="example" style="color: red;
  background-color: pink; font-size: 18px">
  Lorem ipsum dolor sit.
</div>
<div class="example" style="color: red;
  background-color: pink; font-size: 18px">
  Dolores numquam architecto velit.
</div>
```

```
<div class="example" style="color: red;  
    background-color: pink; font-size: 18px">  
    Tempora exsepturi tempore corporis.  
</div>
```

Когда вы работаете с классами, то обычно приходится добавлять или удалять какой-либо класс для одного или нескольких элементов. В примере ниже мы сначала удаляем класс `'active'` для ссылки, которая его имела, а затем назначаем для ссылок, которые находятся внутри `li`-элементов, расположенных в элементе с классом `.menu`.

```
$('a.active').removeClass('active');  
$('.menu li a').addClass('active');
```

Методы для управления стилями и классами в jQuery вы найдете в таблице 4 приложения к данному уроку.

Traversing. Методы обхода DOM

Селекторы в jQuery позволяют обратиться к одному или сразу к нескольким элементам. Причем после этого можно без дополнительного обхода всех элементов в цикле сразу назначать обработчики событий или вызывать нужные методы. Однако в процессе работы с элементами, помимо селекторов, вам понадобится обратиться ко вложенным или родительским элементам селектора, чтобы выполнить ряд действий. Для этого в jQuery существует ряд методов для обхода DOM-элементов (**traversing**). Они представлены в таблице 5 приложения.

В качестве примеров мы будем использовать поиск родительских элементов при клике на кнопке, закрывающей всплывающее окно. В этом случае мы используем метод `closest()`:

```
$('.popup-close').closest('.popup-wrapper').  
fadeOut(400);
```

Для управления скрытым разделом с рекламой самой читаемой книги мы используем метод `parents(selector)`, который ищет указанный селектор среди всех родительских селекторов от текущего.

```
let section = $(this).parents('section');
```

`$(this)` в этом коде — это замена `this` в нативном JavaScript.

При клике на одном из пунктов меню, нам необходимо будет найти в родительском элементе ссылку, у которой есть класс `active` и удалить этот класс. Обращение к родительскому элементу с классом `menu` будет выполнено также с помощью метода `.closest('.menu')`, а поиск нужного элемента — с помощью метода `.find('.active')`. Оба метода принимают в качестве параметра селектор:

```
$(this).closest('.menu').find('.active').  
removeClass('active');
```

Когда мы будем изменять видимость элемента, следующего за активным заголовком, для доступа к этому элементу мы используем метод `next()`:

```
activeHeader.next().slideUp(200);
```

Методы обхода DOM, кроме того, позволяют обрабатывать элементы в другие элементы, перебирать элементы набора, назначая индивидуальные свойства, или обращаться к первому/последнему элементу в наборе.

При разборе примера вы еще увидите их использование.

Обработка событий в jQuery

Как правило, изменения в стилях/классах для html-элементов необходимо назначать при обработке каких-либо событий. Способы обработки событий в jQuery похожи на известные вам из JavaScript, но имеют укороченный синтаксис.

Для того чтобы назначить обработчик события, используйте метод **on**:

```
$('селектор').on('событие', function(){  
    // ваш код  
});
```

Данный синтаксис предполагает, что в качестве события вы указываете любое из известных вам без приставки **'on'**, как для метода **addEventListener()**, например:

```
$('a.active').on('click', function(){  
    $(this).removeClass('active');  
});
```

Вместо ключевого слова **this**, которое в нативном JS указывало на объект, для которого назначено событие,

в jQuery используется аналог — `$(this)`, указывающий на jQuery-объект, соответствующий селектору, но при этом именно тот из набора, по которому был выполнен клик или для которого наступило указанное событие.

Например, для того чтобы получить высоту спрятанного элемента с рекламой наиболее читаемой книги, мы будем обрабатывать событие `resize` для объекта `window` (`$(window)` в jQuery).

```
let sectionMore = $('#see-more'),
    topCoord = $(window).height() -
                sectionMore.outerHeight();

$(window).on('resize', function(){
    topCoord = $(window).height() -
                sectionMore.outerHeight();
    if (!sectionMore.hasClass('closed')) {
        sectionMore.css('top', topCoord);
    }
})
```

Дело в том, что нужный нам элемент — это

```
<section id="see-more" class="closed">
```

Этот раздел находится внизу экрана за счет фиксированного позиционирования и скрыт при загрузке страницы за счет координаты `top: 97vh`.

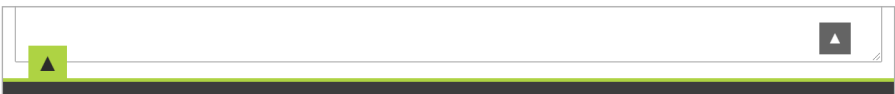


Рисунок 8

При клике на зеленой кнопке со стрелкой мы будем его показывать на всю высоту, которая меняется в зависимости от ширины экрана. За счет обработки события `resize` мы будем динамически получать высоту этого элемента (включая `padding`) методом `sectionMore.outerHeight()` и вычитать ее из высоты окна браузера (`$(window).height()`), а затем назначать результат вычислений для `section` в качестве значения `css`-свойства `top`, если у `section` отсутствует класс `close`, т. е. она находится в развернутом состоянии, как на скриншоте ниже. В этом кусочке кода нам понадобились и методы для управления `css`-стилями, и проверка на существования определенного класса для элемента `section`.

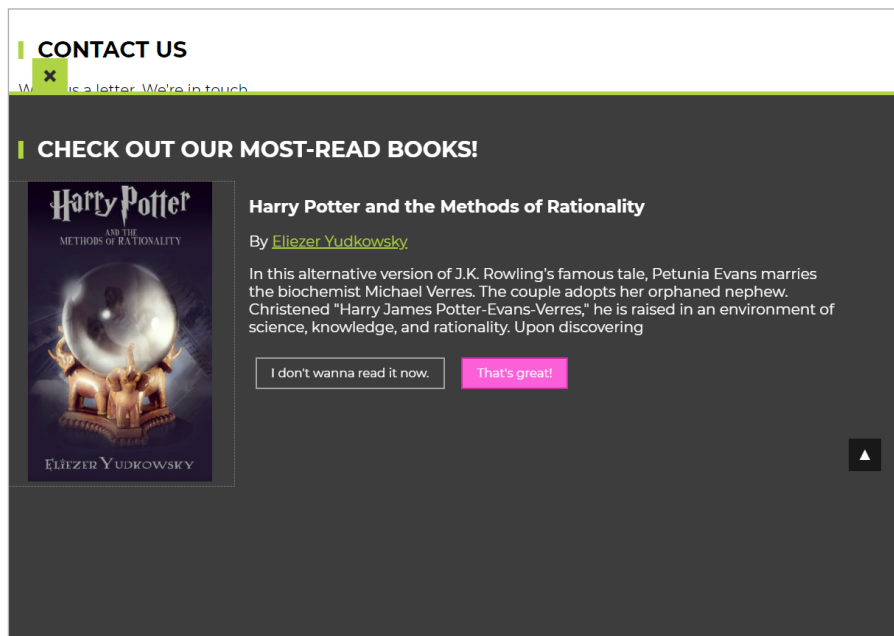


Рисунок 9

В jQuery есть возможность использовать краткую форму обработчика события. Вместо

```
$('селектор').on('mousedown', function(){  });
```

вы можете написать

```
$('селектор').mousedown( function(){  });
```

Естественно, вместо `mousedown` можно использовать события `mouseover`, `mouseout`, `click`, `keyup` или `input`, но без приставки `on`.

Например, для изменения внешнего вида верхнего меню мы можем записать такой код:

```
$('.menu li a').click(function(){
    $(this).closest('.menu').find('.active').
        removeClass('active');
    $(this).addClass('active');
})
```

Что здесь происходит? При загрузке страницы активным пунктом меню является [About Us](#), о чем свидетельствует черная стрелка над текстом ссылки и наличие класса `active` в html-разметке.

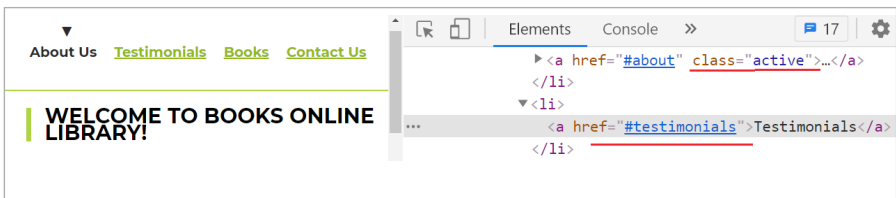


Рисунок 10

Но при клике на любом другом пункте меню, например, на **Testimonials**, черная стрелка переходит к этому пункту вместе с классом **active**, а для **About Us** атрибут **class** становится пустым.

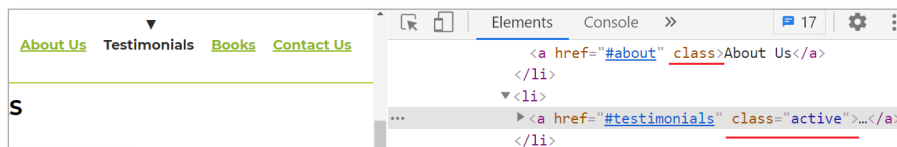


Рисунок 11

В этом коде еще представляет интерес строчка

```
$(this).closest('.menu').find('.active').  
    removeClass('active');
```

В ней мы отталкиваемся от того элемента, по которому был сделан клик (**\$(this)**), причем неважно, какой именно это пункт меню. Затем обращаемся к ближайшему родительскому элементу с классом **.menu** (**closest('.menu')**), находим в нем вложенный элемент с классом **.active** (**find('.active')**), и только потом удаляем у найденного таким сложным способом элемента класс **.active**.

Зачем такие сложности? Почему бы сразу не написать

```
$('.active').removeClass('active');
```

Это можно было бы сделать, если бы у нас не было класса **.active** в других элементах. Однако такой класс присутствует в 2-х элементах в **<div class="accordion">**, поэтому мы можем такой строкой «поломать» работоспособность другого элемента.

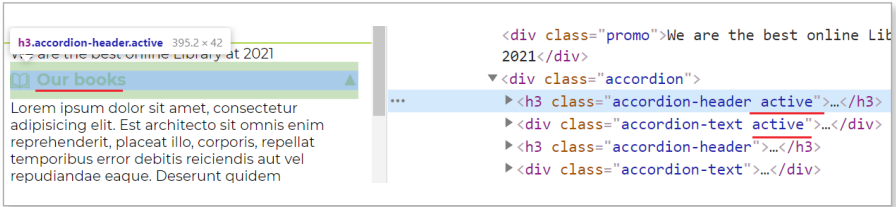


Рисунок 12

Назначать обработчик события можно в виде именованной функции, описанной отдельно, таким образом:

```
$('.menu li a').on('click',toggleMenu);

$('.menu li a').click(toggleMenu);
```

Это имеет смысл делать тогда, когда такая функция вызывается для нескольких элементов в разных блоках кода или не только по событию, но и при загрузке страницы для осуществления какой-либо проверки.

Например, на странице онлайн-библиотеки функция `checkToTop()` проверяет положение полосы прокрутки документа в момент загрузки страницы и по событию `scroll` для `window`, отображая ссылку «To top» только в том случае, если прокручено более `100px` страницы вниз.

```
function checkToTop() {
    $('#totop').css({
        bottom: ($(window).scrollTop() > 100 ? '7%' : '-100px')
    });
}

checkToTop();
$(window).scroll(checkToTop);
```

Разница показана на рисунке 13.

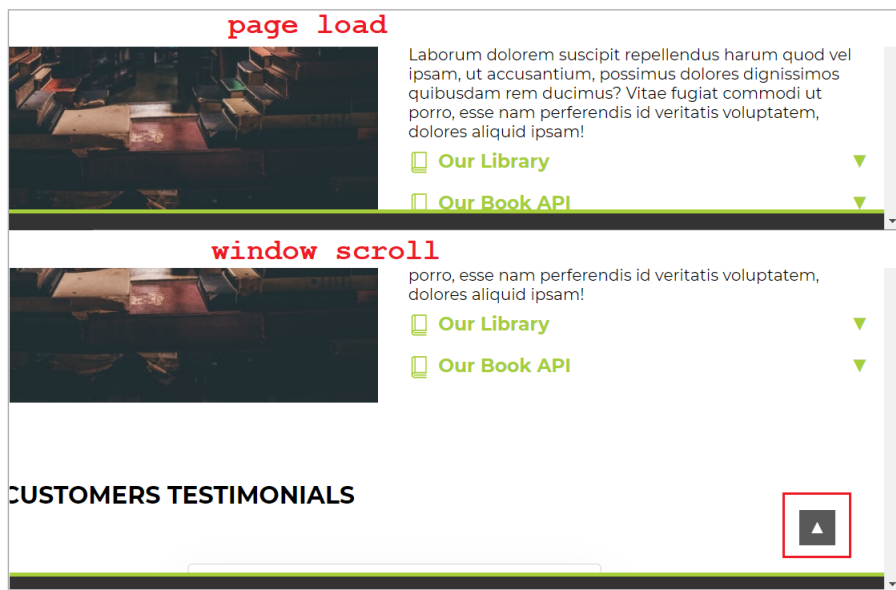


Рисунок 13

Назначение одного обработчика для нескольких событий

Метод `on()` достаточно универсален, и позволяет назначить одну функцию для обработки сразу нескольких событий. Например, при вводе символов в поле ввода можно сразу обработать такие события:

```
$('input:text').on('keydown input paste', function() {
  console.log($(this).val());
});
```

Попробуйте набрать текст в первом поле ввода в разделе [Contact Us](#). Вы увидите лесенку из символов при

ручном наборе и добавление слов(а) при вставке скопированного текста.

`$(this).val()` — это значение **value** для поля ввода.

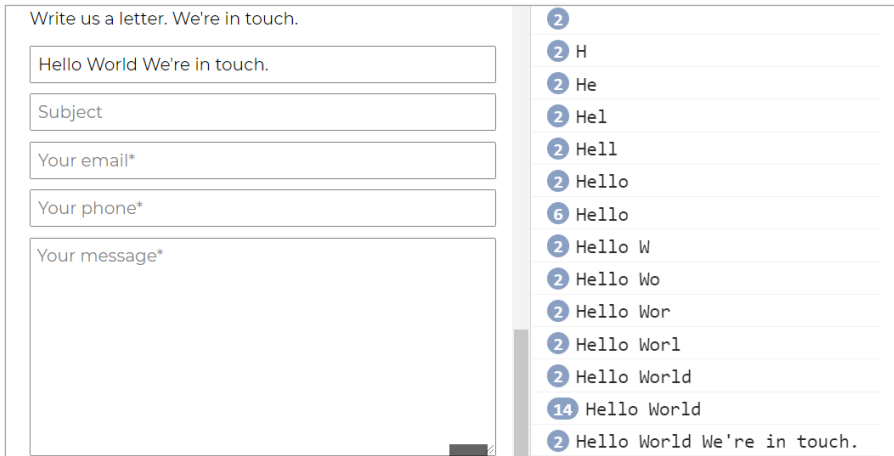


Рисунок 14

Анимационные методы jQuery

В свое время jQuery стала очень популярной не в последнюю очередь из-за того, что позволяла простым способом задавать анимационные эффекты для различных элементов, причем еще в то время, когда css-анимаций с помощью свойств **transition** и **@keyframes** не существовало.

В jQuery анимационные эффекты задаются с помощью нескольких методов. Поскольку для любой анимации необходимо указывать время, то в качестве параметров этих методов используются числа в миллисекундах (`$('.block').hide(500)`) или ключевые слова **'slow'**, **'normal'**, **'fast'**.

К анимационным методам jQuery относятся:

Методы, изменяющие высоту/ширину элемента и его видимость (display)

- `show([time])/hide([time])` — отображение/скрытие элементов с изменением их высоты и ширины во времени. В конце анимации элементу, для которого она вызывалась, добавляется стилевое свойство `display: block` или `display: none`. Параметр `time` является необязательным и позволяет указать время в миллисекундах, в течение которого будет выполняться анимация. Если этот параметр отсутствует, то анимация не выполняется. Происходит только отображение/скрытие элемента.
- `toggle([time])` — метод-переключатель видимости/невидимости объекта. Проверяет, является ли элемент скрытым, и в этом случае показывает его, как метод `show()`. Если элемент отображается, то скрывает его аналогично методу `hide()`.
- `slideUp()/slideDown()` — анимационное скрытие/раскрытие элемента с изменением его высоты.
- `slideToggle()` — метод-переключатель, который скрывает открытый элемент и показывает скрытый элемент с анимационным изменением его высоты. В конце анимации элементу добавляется стилевое свойство `display: block` или `display: none`.

Часть вышеперечисленных методов мы используем для создания аккордеона — элемента, в котором при клике на заголовке отображается спрятанный ниже его текст.

```

$('.accordion-header').on('click', function () {
    let activeHeader = $(this).parent().
        find('.accordion-header.active');
    if (!$ (this).hasClass('active')) {
        activeHeader.next().slideUp(200);
        activeHeader.removeClass('active');
        $(this).addClass('active').next().slideToggle(400);
    }
});

```

Особенностью данного аккордеона является то, что при клике на заголовке меняются стили псевдоэлементов `::before` и `::after` (см. рисунок 15), т. к. к заголовку добавляется класс `active`.

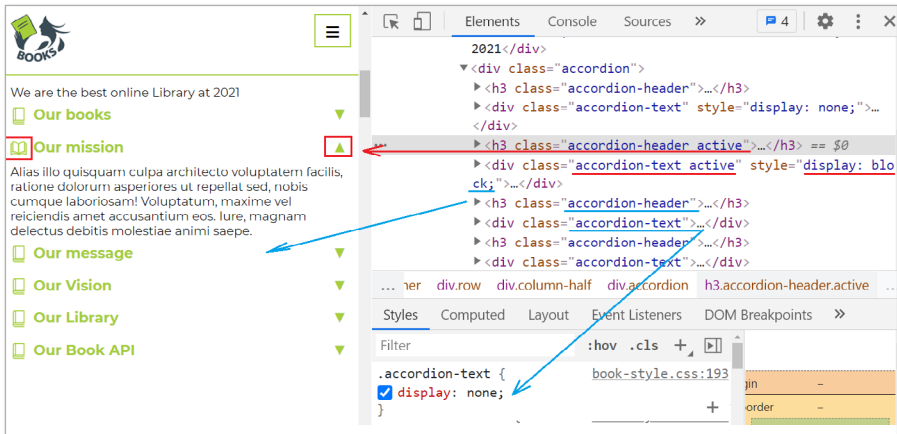


Рисунок 15

Кроме того, при разворачивании одного блока с текстом, открытый ранее блок должен свернуться, так что класс `active` может быть назначен только одному заголовку. Поэтому в коде мы ищем активный заголовок,

используя строку `$(this).parent().find('.accordion-header.active')`, которая выполняет поиск селектора `.accordion-header.active` методом `find()` в родительском по отношению к текущему элементу, которым является `<div class="accordion">`.

Переключать видимость элементов мы будем только для тех, которые на данный момент скрытаны — для этого нам нужна строка `!$(this).hasClass('active')`. Причем нам нужно сначала свернуть раскрытый блок текста, а затем показать новый.

Поскольку тот элемент, который нам нужно скрыть, находится после активного на данный момент заголовка, с точки зрения DOM-модели по отношению к заголовку он будет следующим узлом-элементом, т.е. `nextElementSibling`. С точки зрения jQuery обратиться к следующему узлу можно с помощью метода обхода DOM `next()`:

```
activeHeader.next().slideUp(200);
```

Для разворачивания нового блока текста после другого заголовка, на котором пользователь кликнул, мы используем строку

```
$(this).addClass('active').next().slideToggle(400);
```

За счет того, что при вызове различных методов jQuery возвращается jQuery-объект, мы можем последовательно вызвать метод, добавляющий класс `active` к заголовку, по которому щелкнули, а затем обратиться к следующему за ним элементу и раскрыть его анимационным методом `slideToggle()`.

Методы, изменяющие непрозрачность элемента (свойство *opacity*)

- **fadeIn(time)** — изменяет значение свойства **opacity** от 0 до 1, как бы «проявляя» элемент;
- **fadeOut(time)** — изменяет значение свойства **opacity** от 1 до 0, постепенно скрывая элемент;
- **fadeTo(time, opacityValue)** — изменяет значение свойства **opacity** до требуемого значения (от 0 до 1).

Например, с помощью метода **fadeIn()** через 1,5 секунды после загрузки страницы появляется окно с предложением оформить подписку на сервис.

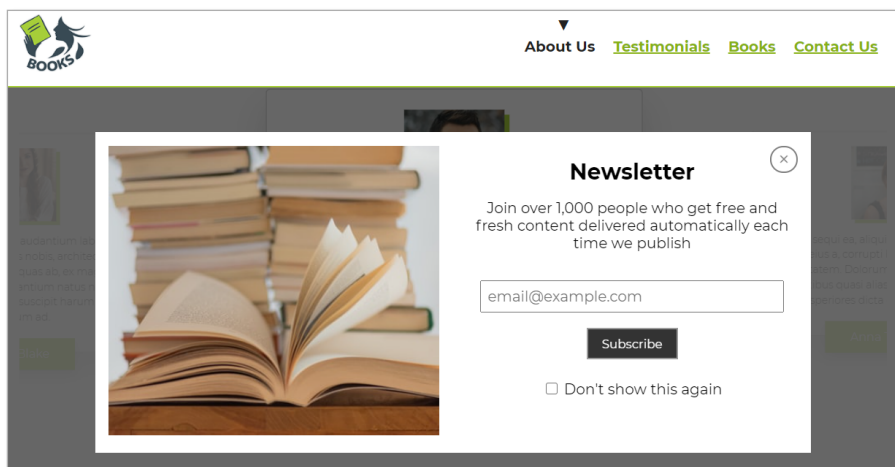


Рисунок 16

Код здесь таков:

```
setTimeout(function () {
    $('#newsletter').fadeIn(500);
}, 1500);
```

Как вы видите, мы вполне успешно сочетаем здесь нативный JavaScript в виде вызова метода `setTimeout()` и анимацию на jQuery.

Для того чтобы закрыть это всплывающее окно, мы должны обработать клик по элементу с классом `.popup-close` и использовать метод `fadeOut()`, который скроет не только окно, но и фон, закрывающий текст страницы.

```
$('.popup-close').click(function () {
    $(this).closest('.popup-wrapper').fadeOut(400);
});
```

За фон отвечает элемент с классом `.popup-wrapper`, который является родителем родительского для `.popup-close` элемента. Поэтому, для того чтобы «добраться» до него, мы используем метод `closest(selector)`, который как раз и возвращает ближайший родительский элемент или текущий элемент с указанным классом.

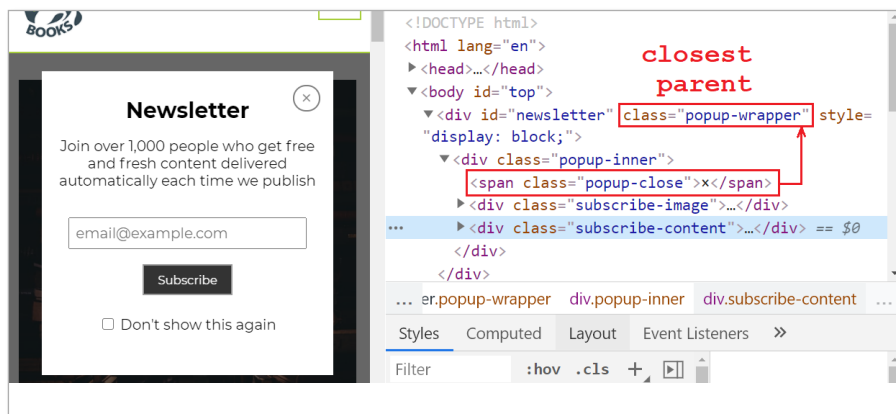


Рисунок 17

Метод `animate()` для создания произвольных эффектов

- `animate({prop1: value, prop2: value}, time)` — позволяет указать объект, содержащий свойства и значения любых свойств, которые можно изменить количественно, то есть `border-radius`, `width`, `height`, `left`, `top`, `padding`, `margin` и т. п. и количество времени на анимацию. Вы не можете с помощью этого метода анимировать такие свойства, как `visibility` или `display`, а также `background-color` или `color`. Для анимации последних существует плагин [jQuery.Color](#).

Рассмотрим примеры с нашей страницы, которые используют метод `animate()` при обработке событий. Например, клик по зеленой кнопке-стрелке внизу экрана отобразит нам блок с рекламой самой читаемой книги:

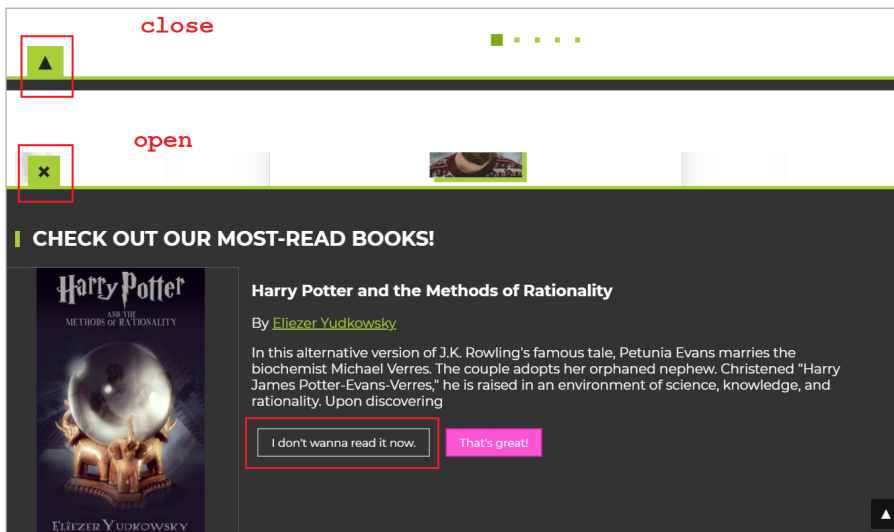


Рисунок 18

Эта кнопка имеет разный вид, когда раздел страницы с ней скрыт и полностью виден. Кроме того, скрывать его мы будем еще и при клике на кнопке с `id="not-now"` и текстом *I don't wanna read it now*. Поэтому обработчик события запишем для 2-х селекторов сразу, а в функции определим переменную `section` для обращения к родительскому элементу с соответствующим селектором тега, а переменную `target` — для ссылки на зеленую кнопку. Метод `animate()` нам необходим для того, чтобы красиво передвинуть верх фиксированной `section` в рассчитанную ранее координату или опустить ее в самый низ. Кроме того, для изменения вида зеленой кнопки мы используем метод jQuery `html()`, передавая в него спецсимволы, обозначающие стрелку или крестик в зависимости от того, открыта или нет наша `section`. За это отвечает наличие / отсутствие для раздела с рекламой класса `closed`, который добавляется / удаляется в коде строкой `section.toggleClass('closed')`.

```
$('.close-green, #not-now').click(function () {
  let section = $(this).parents('section');
  let target = $('.close-green');
  if (section.hasClass('closed')) {
    section.animate({
      top: topCoord
    }, 400);
    target.html('&times;');
  } else {
    section.animate({
      top: '97vh'
    }, 400);
  }
});
```



```

        target.html('&#9650;');
    }

    section.toggleClass('closed');
});

```

Еще один вариант использования метода `animate()` предполагает анимированную прокрутку до нужного раздела страницы или на самый верх при клике на ссылке с хешем (`#`):

```

$('[href^="#"]').on('click', function (event) {
    if ($(this).attr('hash') !== "") {
        event.preventDefault();
        let hash = $(this).prop('hash');
        $('html, body').animate({
            scrollTop: $(hash).offset().top - 60
        }, 800, function () {});
    }
});

```

Здесь мы анимируем для `html` и `body` такое свойство, как `scrollTop` элемента, получая его текущее положение элемента относительно верха страницы.

Делегирование событий

Делегирование событий подразумевает, что обработчик события назначается для родительского элемента, а само событие обрабатывается для вложенных (дочерних) элементов. Этот вариант бывает необходим тогда, когда вложенных элементов очень много или эти элементы были добавлены в процессе работы JS или jQuery-кода.

Рассмотрим пример, в котором при наведении на элементы списка мы должны получить в блоке, расположенном справа от них, какую-то английскую поговорку. Текст в элементах списка нам дает только примерное представление о сути поговорки. Сама поговорка скрыта в атрибуте `data-text` внутри элемента `<li>`:

```
<ul id="proverbs">
  <li data-text="When in Rome, do as the Romans.">
    How to behave...
  </li>

  <li data-text="Hope for the best,
                but prepare for the worst.">
    About hope
  </li>

  <li data-text="Fortune favors the bold.">
    About fortune
  </li>

  <li data-text="Birds of a feather flock together.">
    About people
  </li>

  <li data-text="Keep your friends close
                and your enemies closer.">
    About friends and enemies
  </li>
</ul>
```

Мы будем обрабатывать 2 события — `mouseenter` и `mouseleave` для родительского элемента `<ul>`, но при этом получать информацию о поговорках будем из элементов `<li>`, используя свойство `event.target` (селектор `$(event.target)` в jQuery) объекта `event`, который всегда

передается в качестве параметра в функцию-обработчик события. Сделаем мы это потому, что используем в этом скрипте синтаксис стрелочных функций из стандарта ES6(EcmaScript2015).

```
$(()=>{
  const output = $('#output');
  $('ul#proverbs').on('mouseenter', 'li', (event)=>{
    console.log($(event.target));
    output.text($(event.target).data('text'));
  }).on('mouseleave', 'li', (event)=>{
    output.text('');
  });
})
```

Код предполагает, что при обработке события `mouseenter` мы выводим в пустом `<div id="output">` поговорку из `<li>`, используя метод `data('text')` для доступа к атрибуту `data-text`, а при обработке события `mouseleave` убираем текст, помещая в `#output` пустую строку.

В строке `console.log($(event.target));` выводится информация о том, что мы действительно обращаемся к элементу `<li>`. Пример вы можете найти в папке *examples* в файле *delegate.html*.

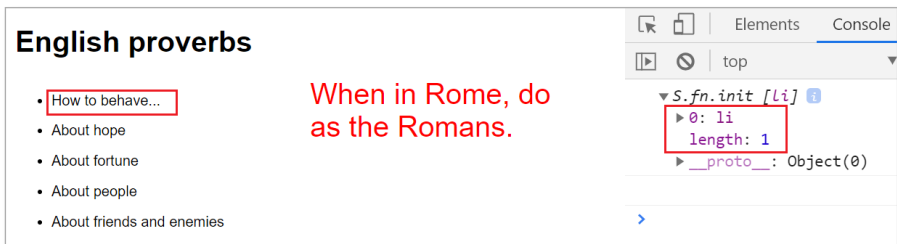


Рисунок 19

В jQuery делегирование событий осуществляется методом `on()`. Общий вид таков:

```
$('родительский селектор').  
  on('событие', 'селектор дочерних элементов',  
    function(){ ... });
```

Например, для элементов типа `<button class = "btn-plus">`, которые могут добавляться в форму путем клонирования обработка события клика будет такой:

```
$('form').on('click', '.btn-plus', function(){  
  console.log('button clicked');  
});
```

В нашем примере такое действие происходит для открытия всплывающих окон. Таких в разметке страницы 2: для подписки на новости

```
<div id="newsletter" class="popup-wrapper">  
  ...  
</div>
```

и для вывода сообщения при заполнении формы контактов

```
<div id="thanks-contact" class="popup-wrapper">  
  ...  
</div>
```

Всплывающее окно с подпиской выводится либо через 1,5 секунды после загрузки страницы (мы разбирали этот пункт выше), либо при клике на кнопку в элементе `<section id="subscribe">` (рис. 20).

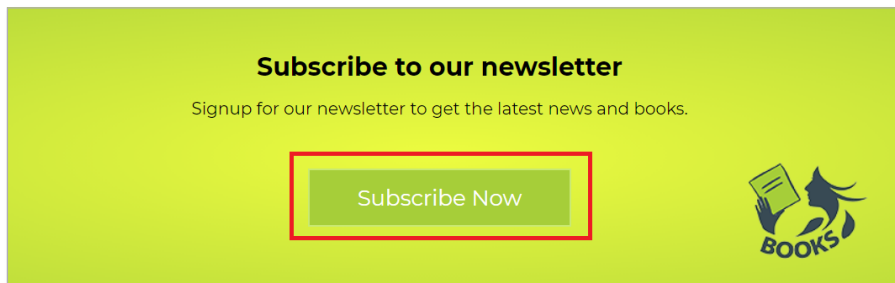


Рисунок 20

Код кнопки:

```
<button class="btn btn-info btn-large"
        data-popup="#newsletter">
    Subscribe Now
</button>
```

Как видно, у этой кнопки есть data-атрибут `data-popup="#newsletter"`, который мы и примем за основу при назначении обработчика события в виде функции `popupShow()`;

```
$('[data-popup]').on('click', popupShow);

function popupShow(evt) {
    evt.preventDefault();
    let id = $($this).data('popup');
    id.fadeIn(600);
};
```

В функции мы получаем `id` элемента, указанного в атрибуте `data-popup` и с помощью анимационного метода `fadeIn(600)` показываем соответствующее всплывающее окно:

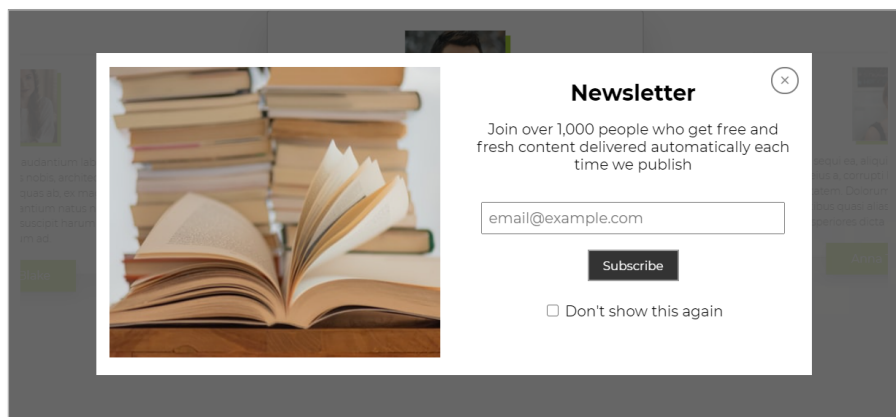


Рисунок 21

Где же здесь делегирование события? Его действительно пока нет. Дело в том, что второе всплывающее окно мы вызываем после проверки правильности заполнения полей формы, причем атрибут `data-popup="#thanks-contact"` добавляется в скрипте после проверки всех полей (см. описание процесса ниже). А это говорит о том, что на момент загрузки страницы кнопка с кодом

```
<button type="submit" id="contact-btn"
        class="btn btn-dark">
  Send a letter
</button>
```

не имела назначенного обработчика события, т. к. в ее разметке отсутствует нужный data-атрибут, поэтому функция `popupShow()`, как обработчик события, просто не будет вызвана.

Как раз для этой ситуации и нужно воспользоваться делегированием событий. Мы назначим обработчик со-

бытия для элемента `body`, однако отслеживать его будем только для элементов с атрибутом `data-popup`. Записать это нужно таким образом:

```
$('body').on('click', '[data-popup]', popupShow);
```

вместо строки

```
// $('[data-popup]').on('click', popupShow);
```

Пользователь, заполнивший поля формы и нажавший на кнопку с текстом «Send a letter», увидит примерно такое сообщение:

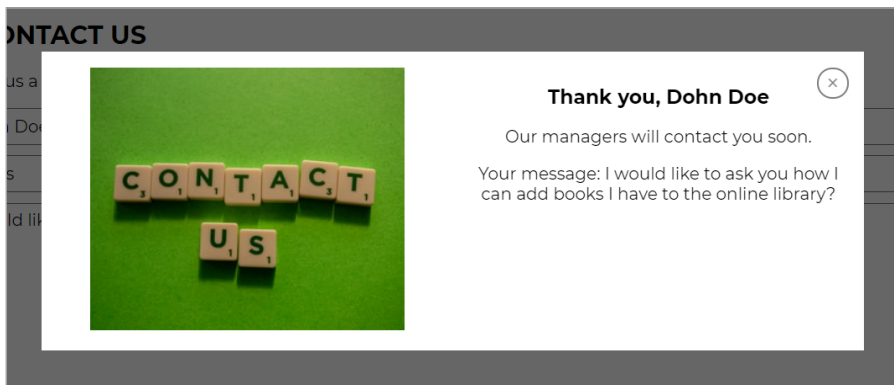


Рисунок 22

Отмена обработчиков событий

В том случае, когда нам в какой-то момент нужно, чтобы обработчик события был удален, применяется метод `off()`. При использовании этого метода важно, чтобы при назначении обработчика события использовалась та же функция, что и для его отмены, как для методов

`addEventListener()` и `removeEventListener()` в нативном JavaScript. Общий вид записи:

```
//назначаем обработчик события
$('селектор').on('событие', someFunc);

//отменяем обработчик события
$('селектор').off('событие', someFunc);
```

Например, такой подход мы используем для маленького рекламного блока в **header** нашей страницы. По центру вы можете видеть зеленый восклицательный знак — это кнопка с `id="start-stop"`. Рядом с ней в разметке расположен `<span id="text-change"></span>`, куда при нажатии на эту кнопку будет выводиться несколько строк из массива с помощью jQuery. Внешний вид кнопки будет меняться при этом на крестик, и повторное нажатие на кнопку приведет к остановке смены текста и его удалению из header-a. То есть кнопка будет переключателем работы 2-х функций.

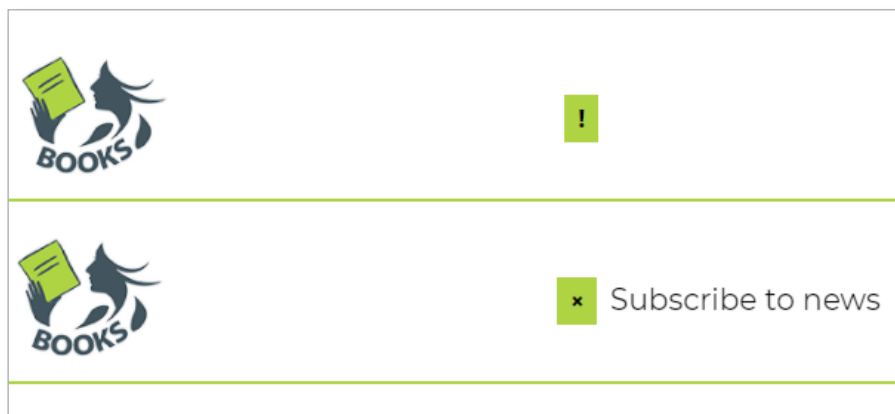


Рисунок 23

В коде это будет выглядеть так:

```
let textAnim = ['Read our books!', 'Subscribe to news',
               'Learn about new books',
               'Add books to our library'],
    textId=null, textIndex=0;
$("#start-stop").on('click', showAnimatedText);

function showAnimatedText(){
    $(this).next().addClass('fromTopToDown');
    textId = setInterval(function(){
        $('#text-change').text(textAnim[textIndex]);
        textIndex == textAnim.length-1 ?
            textIndex = 0: textIndex++;
    }, 2000);
    $("#start-stop").html('&times;')
    .off('click', showAnimatedText)
    .on('click', stopAnimatedText);
}

function stopAnimatedText(){
    $('#text-change').text('');
    $(this).next().removeClass('fromTopToDown');
    clearInterval(textId);
    $("#start-stop").text('!')
    .off('click', stopAnimatedText)
    .on('click', showAnimatedText);
}
```

Анимацию смены текста в данном случае мы реализуем за счет добавления/удаления к текстовому блоку класса **fromTopToDown** с такими CSS-свойствами:

```
.fromTopToDown {
    animation: fromTopToDown 2s ease-out infinite;
}
```

Метод `off()` также позволяет отменить события, назначенные с использованием делегирования события. В этом случае запись будет такая:

```
$('родительский селектор').off('событие',  
    'селектор дочерних элементов', someFunc);
```

Обработка данных форм

Для форм в арсенале jQuery есть свои селекторы (таблица 3 приложения). В основном, они помогают найти однотипные селекторы для различных вариантов полей `input` или `button` (`$(':submit')`, `$(':input')`), а также обратиться к выделенным флажкам или переключателям (`$(':checked')`) или выбранным пунктам выпадающих списков (`$('option:selected')`).

Флажок (`checkbox`) встречается нам на странице 1 раз во всплывающем окне с предложением оформить подписку. Он имеет в коде `id="forgetMe"`. Клик по этому элементу предполагает, что при следующем заходе на страницу мы уже не увидим данное окно.

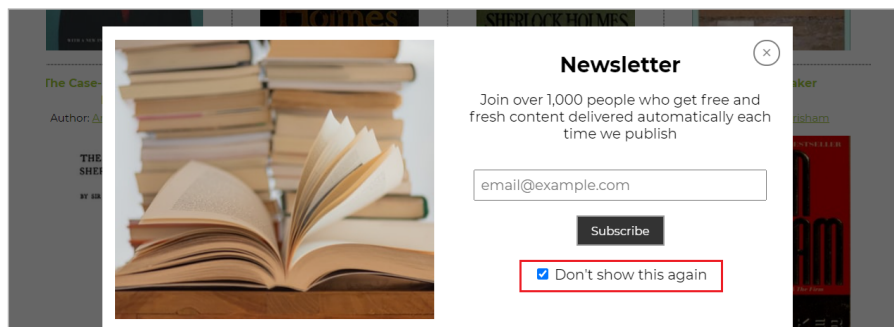


Рисунок 24

Поэтому обрабатываем для флажка событие `click`, сохраняя в `localStorage` браузера информацию о том, что пользователь выделил флажок. Объекте `localStorage` хорош тем, что он сохраняет нужные данные даже после того, как пользователь закрыл страницу в браузере, так что при следующем заходе на этот сайт код на JavaScript (jQuery) может проверить эти данные и отображать информацию для этого конкретного пользователя, основываясь на его предпочтениях.

Для этого действия нам понадобится такое свойство флажка, как `checked`, которое при обработке события мы можем получить строкой `$(this).prop('checked')`. Если оно возвращает `true`, то в `localStorage` в виде значения `'newsletter'` мы помещаем `1`, если же пользователь снял такую отметку и свойство стало равно `false`, то сохраняем в `newsletter` `0`.

```
$('#forgetMe').click(function () {  
    console.log($(this).prop('checked'));  
    localStorage.setItem('newsletter',  
        $(this).prop('checked') ? 1 : 0);  
});
```

Кроме того, мы теперь несколько изменим код появления самого всплывающего окна: перед вызовом `setTimeout()` сначала идет проверка на наличие в `localStorage` данных с именем `newsletter`, и только в случае, если таких данных там нет или там находится `0`, окно будет показано через `1,5` секунды после загрузки страницы.

```
let isNewsletterPopup = +localStorage.  
    getItem('newsletter');
```

```

if (!isNewsletterPopup) {
  setTimeout(function () {
    $('#newsletter').fadeIn(500);
  }, 1500);
}

```

В нижней части страницы у нас есть контактная форма. Для нее мы будем обрабатывать событие `submit`. Однако, до того как пользователь будет отправлять форму, мы используем плагин [jquery.maskedinput](#) для того, чтобы пользователь вводил в поле для телефона только цифры и в определенной последовательности:

```

let userPhone = $('#user-phone');
userPhone.mask("(999) 999-9999");

```

Чтобы плагин сработал, вам необходимо его подключить после jQuery, но до вашего файла с основным скриптом для страницы.

```

<script src="https://ajax.googleapis.com/ajax/libs/
  jquery/3.5.1/jquery.min.js">
</script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/
  jquery.maskedinput/1.4.1/jquery.
  maskedinput.min.js">
</script>
<script src="js/index.js"></script>

```

Этот код приведет к тому, что при попадании фокуса в поле с текстом «Your phone» пользователь увидит подсказку в виде подчеркиваний в месте расположения цифр.



Рисунок 25

Как вы видите, использование этого плагина очень простое. Больше вариантов оформления маски вы найдете на [странице плагина](#).

Для остальных полей перед отправкой формы мы выполним элементарную проверку на соответствие определенному количеству символов. Если символов недостаточно, мы будем подсвечивать красной рамкой соответствующее поле ввода (за это отвечает класс `invalid` в CSS-стилях). Если же пользователь исправил свои ошибки, то этот класс мы будем убирать.

Основой для проверки служит вызов метода `.val()`, который в jQuery служит заменой свойству `value` для элементов форм в нативном JavaScript.

```
$('#contact-form').submit(function (e) {
    e.preventDefault();
    let userName = $('#user-name'),
        userEmail = $('#user-email'),
        userMessage = $('#user-message');

    // проверяем заполнение полей формы
    if (userName.val().length < 2) {
        userName.addClass('invalid');
        return false;
    } else userName.removeClass('invalid');

    if (userEmail.val().length < 7) {
        userEmail.addClass('invalid');
```

```

        return false;
    } else userEmail.removeClass('invalid');

    if (userPhone.val().length < 7) {
        userPhone.addClass('invalid');
        return false;
    } else userPhone.removeClass('invalid');

    if (userMessage.val().length < 7) {
        userMessage.addClass('invalid');
        return false;
    } else userMessage.removeClass('invalid');

    // помещаем текст с сообщением в popup-окно
    // и показываем его пользователю

    $('#thanks-contact h3').text('Thank you,
        ${userName.val()}');
    $('#thanks-contact .content').
        text('Your message: ${userMessage.val()}');
    $('#contact-btn').attr('data-popup',
        '#thanks-contact');
    $('#contact-btn').click();

    })

```

После того, как пользователь все заполнил верно и щелкнул на кнопку «Send a letter», мы выводим его имя и текст сообщения в модальное окно с `id="thanks-contact"` с помощью метода `jQuery.text()`. Кроме того, добавляем `attr()` для кнопки `submit` атрибут `data-popup`, который отвечает за запуск всплывающего окна, со значением «`#thanks-contact`», а потом еще и генерируем клик по этой кнопке, чтобы увидеть окно с текстом.

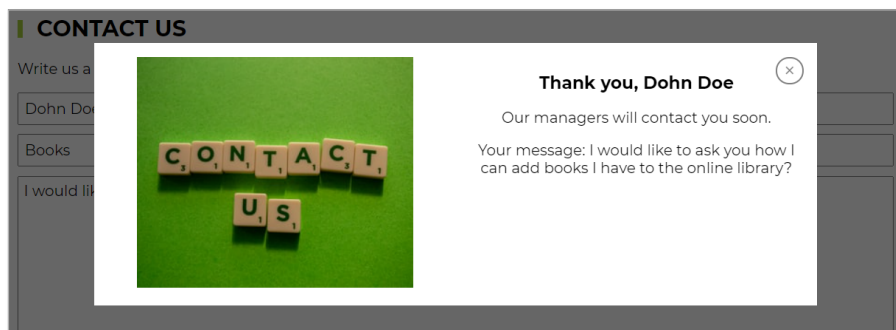


Рисунок 26

jQuery-плагины

Информация о jQuery была бы неполной, если не упомянуть о различных плагинах, которые были написаны сообществом разработчиков и предлагаются для бесплатного скачивания. С одним таким плагином — `jquery.maskedinput` — мы уже познакомились при обработке элементов форм. Плагины под jQuery решают различные задачи — от построения различных анимированных графиков, создания модальных окон до прокрутки блоков экранами. Больше всего плагинов, пожалуй, создано для фотогалерей и слайдеров. Именно из этой группы мы и будем рассматривать слайдер для формирования отзывов пользователей. Называется он «Owl Carousel» и доступен для скачивания на [официальном сайте](#). Он хорош тем, что позволяет настроить отображение элементов под различные экраны, поддерживает события перетаскивания для различных браузеров (современных и не очень) и весит порядка 50кб. Плюс этот плагин можно не только скачать, но и подключить к вашей странице через CDN, что мы и сделаем.

Общая схема работы с jQuery-плагинами обычно укладывается в такую последовательность:

1. Подключаем в блоке `<head>` стилевой файл плагина с помощью тега `<link>`. Например, для Owl Carousel:

```
<!-- Из локальной папки css -->
<link rel="stylesheet"
      href="css/owl.carousel.min.css">

<!-- либо с CDN -->
<link rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/
            OwlCarousel2/2.3.4/assets/owl.carousel.css">
```

2. Добавляем определенную разметку в html-файл.

```
<div id="customers-testimonials" class="owl-carousel">
  <div class="item">
    <div class="shadow-effect">
      
      <p>Lorem ipsum dolor sit... animi.</p>
    </div>
    <div class="testimonial-name">
      John Doe
    </div>
  </div>
  <div class="item">
    <div class="shadow-effect">
      
      <p>Quasi sequi ea, aliquid ... repellat.</p>
    </div>
```



```

        <div class="testimonial-name">
            Anna Traube
        </div>
    </div>
    ...
</div>

```

3. В нижней части `<body>` подключаем js-файл плагина после jQuery:

```

<!-- Сначала подключаем js-файл с jQuery -->
<script src="https://ajax.googleapis.com/ajax/libs/
    jquery/3.5.1/jquery.min.js">
</script>

<!-- owl carousel из локальной папки js -->
<script src="js/owl.carousel.min.js"></script>

<!-- либо с CDN -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/
    OwlCarousel2/2.3.4/owl.carousel.min.js">
</script>

```

4. Подключаем скрипт js-файла, в котором будет вызвана основная функция плагина:

```

<script src="js/index.js"></script>

```

5. Вызываем функцию плагина с настройками:

```

$('#customers-testimonials').owlCarousel({
    loop: true,
    center: true,
    items: 3,
    margin: 0,
    // dots: true,

```

```
// autoplay: true,
// autoplayTimeout: 8500,
smartSpeed: 450,
responsive: {
  0: {
    items: 1
  },
  800: {
    items: 3
  }
}
});
```

Настройки Owl Carousel вы можете найти в [документации на официальном сайте](#). Здесь приведем описание тех настроек, которые были использованы в примере:

- **loop** — зацикливание карусели. Происходит дублирование первого и последнего элементов, чтобы получить иллюзию постоянно прокручиваемой карусели;
- **center** — центрирование элементов. Опция хорошо работает как с четным, так и с нечетным количеством элементов;
- **items** — количество элементов, отображаемых в карусели;
- **margin** — свойство **margin-right** (в **px**) для каждого элемента;
- **dots: true** — показывает навигацию с помощью точек внизу карусели (значение по умолчанию);
- **autoplay: true** — позволяет прокручивать карусель автоматически через определенные промежутки времени.

В примере эта опция закомментирована, но вы можете удалить комментарий и посмотреть, как изменится работа карусели.

- **autoplayTimeout: 8500** — время в миллисекундах, через которое происходит автоматическая смена (прокручивание) слайдов в карусели. Раскомментируйте опцию в примере, если хотите, чтобы карусель прокручивалась автоматически через 8,5 секунд.
- **smartSpeed** — скорость смены слайдов в миллисекундах при клике навигационные точки.
- **responsive** — возможность задать гибкие (адаптивные) настройки для различных разрешений экрана в виде объекта, для которого разработчики выдали [такие рекомендации](#):
 - Каждый ключ точки останова может быть числовым значением (480) или строкой: '480'.
 - Owl имеет встроенную опцию сортировки, но лучше всего установить от самых маленьких экранов до самых широких.
 - Адаптивные параметры всегда перезаписывают настройки верхнего уровня.
 - По умолчанию для параметра **responsive** установлено значение **true**, поэтому карусель всегда пытается соответствовать оболочке (даже если медиа-запросы не поддерживают IE7 / IE8 и т. д.).
 - Если у вас негибкий макет, установите **responsive: false**.

В нашем примере мы изменяем для этой настройки количество отображаемых слайдов с 3 для больших экра-

нов до 1 для экранов планшетов и смартфонов. Остальные настройки вы можете разобрать самостоятельно.

Имейте в виду, что для оформления карусели был добавлен CSS-код в блоке с комментарием `/*-- Testimonials --*/`:

```
.shadow-effect {
  background: #fff;
  padding: 20px;
  border-radius: 4px;
  text-align: center;
  border: 1px solid #d6d6d6;
  box-shadow: 0 19px 38px rgba(0, 0, 0, 0.10),
              0 15px 12px rgba(0, 0, 0, 0.02);
}

.shadow-effect p {
  font-size: 14px;
  line-height: 1.5;
  margin: 0 0 17px 0;
  font-weight: 300;
}

.testimonial-img {
  box-shadow: 5px 5px 0 #a6ce39;
  max-width: 100px;
  margin: 0 auto 17px;
}

.testimonial-name {
  margin: -20px auto 0;
  display: table;
  width: auto;
  background: #a6ce39;
  padding: 12px 35px;
  text-align: center;
  color: #fff;
  box-shadow: 0 9px 18px rgba(0, 0, 0, 0.12),
              0 5px 7px rgba(0, 0, 0, 0.05);
}
```

```

#customers-testimonials .item {
  text-align: center;
  padding: 50px;
  margin-bottom: 80px;
  opacity: .2;
  transform: scale(0.8);
  transition: all 0.3s ease-in-out;
}

#customers-testimonials .owl-item.active.center .item
{
  opacity: 1;
  transform: scale(1.10);
}

#customers-testimonials.owl-carousel .owl-dots
      .owl-dot.active span,
#customers-testimonials.owl-carousel .owl-dots
      .owl-dot:hover span {
  background: #82a819;
  transform: translateY(-50%) scale(0.7);
}

#customers-testimonials.owl-carousel .owl-dots {
  display: inline-block;
  width: 100%;
  text-align: center;
}

#customers-testimonials.owl-carousel .owl-dots .owl-dot {
  display: inline-block;
  outline: none;
}

#customers-testimonials.owl-carousel .owl-dots
      .owl-dot span {
  background: #a6ce39;
  display: inline-block;
  height: 20px;
  margin: 0 2px 5px;
}

```

```
transform: translateY(-50%) scale(0.3);  
transition: all 250ms ease-out;  
width: 20px;  
}
```

jQuery и AJAX

jQuery предоставляет очень простые методы для работы с технологией AJAX. Эта технология предназначена для асинхронной загрузки и передачи данных между сервером и html-страницей посредством JavaScript. То есть данные, которые мы можем отображать на странице, изначально на ней не присутствовали, однако AJAX позволяет их получить из внешнего файла. Либо мы можем отправить данные любой формы и без перезагрузки страницы получить ответ от сервера, например, о том, что на почту пользователю отправлено письмо с запрошенной информацией.

Мы будем загружать с помощью AJAX и jQuery список книг из библиотеки, предоставляющей информацию о книгах в виде открытого API, который к тому же не требует регистрации.

API (*Application Programming Interface*), или программный интерфейс приложения, позволяет пользователям какого-либо сайта получать информацию о продуктах/услугах, представляемых сайтом, или возможность совершать какие-то действия, например, проводить платежи.

Для нашей страницы нужна информация о книгах, которая предоставлена на сайте <https://openlibrary.org/developers/api> в виде различных вариантов: [Books](#), [Covers](#),

[Lists](#), [Read](#), [Recent Changes](#), [Search](#), [Search inside](#) и [Subjects](#). Мы используем возможность загружать книги по такому параметру, как Subject (Рубрика — <https://openlibrary.org/dev/docs/api/subjects>), и выберем в качестве рубрики [Mystery and detective stories](#). Все рубрики OpenLibrary вы найдете по адресу <https://openlibrary.org/subjects>.

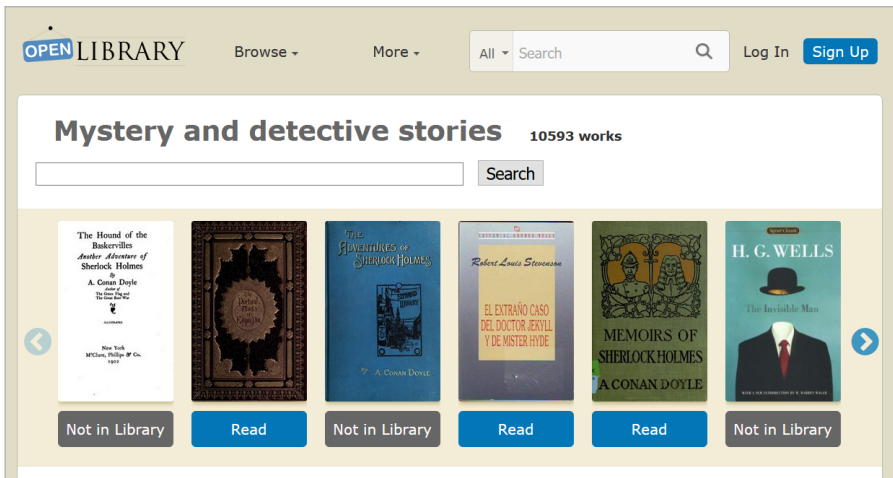


Рисунок 27

API OpenLibrary Subject подразумевает, что вы можете загрузить JSON-файл, в котором будет находиться данные о 12 (или меньшем количестве) книгах в указанной рубрике. Нужный нам этот файл будет находиться по адресу https://openlibrary.org/subjects/mystery_and_detective_stories.json.

Теперь нам нужно разобраться с тем, как с помощью методов jQuery выполнить асинхронную загрузку книг с помощью технологии AJAX. Для этого нам понадобится метод [\\$.ajax\(\)](#), который предполагает такую запись:

```
$.ajax({
  url: 'somefile.ext',
  dataType: "json",
}).done(function(data) {
  // код, который работает в случае успешной
  // отправки формы на сервер
  console.log("Ok! " + data);
}).fail(function() {
  // код, который будет выводиться в случае ошибки
  console.log("Error from mail!!!" + data);
});
```

В самом методе `$.ajax()` можно указать довольно много опций, но параметр `url` является обязательным, а `dataType: "json"` важен для загрузки данных в соответствующем формате. Поскольку мы будем запрашивать данные методом `GET`, то еще один важный параметр `method` мы не указываем, т. к. по умолчанию он имеет значение `'GET'`. Если же мы будем отправлять данные, то тогда нам понадобится явно задать параметр `method: 'POST'`.

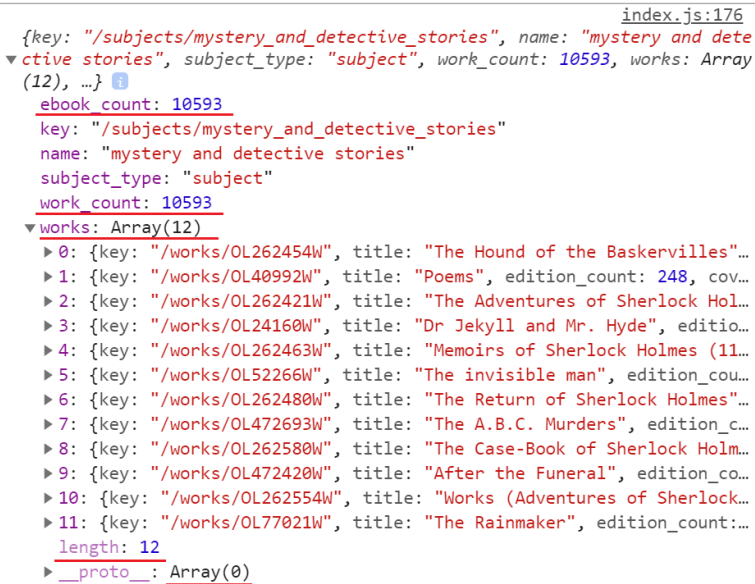
Метод `$.ajax()` посылает запрос на указанный в `url` адрес и получает ответ в виде данных, которые за счет параметра `dataType: "json"` представлены не в виде текста, а в виде объекта или массива объектов. В случае успешной передачи данных возвращается ответ (`response`) и обрабатывается в функции `done()`. Если запрос завершился ошибкой, то обработка ошибки выполняется функцией `fail()`.

Пока что мы поработаем с функцией `done()` и выведем в `console.log()` все данные, полученные с сервера `OpenLibrary` в переменную `data`:


```

var category_url = "https://openlibrary.org/subjects/
                    mystery_and_detective_stories";
let bookDiv = '';
$.ajax({
  url: category_url,
  dataType: "json"
}).done(function (data) {
  console.log(data);
})

```



```

index.js:176
{key: "/subjects/mystery_and_detective_stories", name: "mystery and dete
▼ ctive stories", subject_type: "subject", work_count: 10593, works: Array
(12), ...}
  ebook_count: 10593
  key: "/subjects/mystery_and_detective_stories"
  name: "mystery and detective stories"
  subject_type: "subject"
  work_count: 10593
  works: Array(12)
    ► 0: {key: "/works/OL262454W", title: "The Hound of the Baskervilles"...
    ► 1: {key: "/works/OL40992W", title: "Poems", edition_count: 248, cov...
    ► 2: {key: "/works/OL262421W", title: "The Adventures of Sherlock Hol...
    ► 3: {key: "/works/OL24160W", title: "Dr Jekyll and Mr. Hyde", editio...
    ► 4: {key: "/works/OL262463W", title: "Memoirs of Sherlock Holmes (11...
    ► 5: {key: "/works/OL52266W", title: "The invisible man", edition_cou...
    ► 6: {key: "/works/OL262480W", title: "The Return of Sherlock Holmes"...
    ► 7: {key: "/works/OL472693W", title: "The A.B.C. Murders", edition_c...
    ► 8: {key: "/works/OL262580W", title: "The Case-Book of Sherlock Holm...
    ► 9: {key: "/works/OL472420W", title: "After the Funeral", edition_co...
    ► 10: {key: "/works/OL262554W", title: "Works (Adventures of Sherlock...
    ► 11: {key: "/works/OL77021W", title: "The Rainmaker", edition_count:...
    length: 12
    __proto__: Array(0)

```

Рисунок 28

После обновления страницы вы увидите в консоли объект с множеством полей, из которых для нас будут представлять интерес `ebook_count` и `work_count`, которые содержат число, указывающее на количество книг в данной рубрике (важно, чтобы было больше 0), а также

поле `works`, в котором мы видим массив из 12 элементов (рис. 28). Это то, что нам нужно — информация о книгах.

Поскольку поле (свойство) `works` — это массив, то для вывода книг нам нужно его перебрать и на основе данных в каждом элементе массива создать `div`, отвечающий за внешний вид блока с информацией о книге. Перебирать массив в JavaScript удобно с помощью метода `forEach()`, в jQuery для этого служит специальный метод `$.each()`, который имеет такой синтаксис:

```
$.each(arr, function(index, item){
    console.log(index, item); })
```

где:

`arr` — это массив, который мы перебираем;

`index` — индекс элемента;

`item` — значение элемента.

Нам еще понадобится переменная `bookDiv`, которая сначала будет пустой строкой и в которую мы потом запишем всю разметку для книг.

Перепишем наш код для массива `data.works`:

```
let bookDiv = '';
$.ajax({
    url: category_url,
    dataType: "json",
    beforeSend: function () {
        $('#loader').show();
    }
}).done(function (data) {
    // console.log(data);
    $('#loader').hide();
```

```

$.each(data.works, function (index, bookInfo) {
    console.log(bookInfo);

    if (bookInfo.authors.length == 1)
        var authorsInfo = 'Author: <a href=
            "https://openlibrary.org/${bookInfo.
            authors[0].key}/">${bookInfo.authors[0].
            name}</a>';
    else {
        var authorsInfo = 'Authors: ';
        $.each(bookInfo.authors, function (i, author) {
            authorsInfo += '<a href="https://
                openlibrary.org/${author.
                key}/">${author.name}</a>';
        });
        authorsInfo = authorsInfo.slice(0, -2);
    }

    bookDiv += '<div class="book">
        <div class="book-title">
            ${bookInfo.title.length < 50 ?
            bookInfo.title :
            bookInfo.title.slice(0, 60) + '...'}
        </div>
        <div class="book-author">
            ${authorsInfo}
        </div>
        <div class="book-cover">
            
        </div>
        <div class="book-hidden">
            <a href="https://openlibrary.org${bookInfo.key}"
                class="btn btn-info" target="_blank">
                More Info
            </a>
    </div>

```

```

    ${bookInfo.availability.is_readable ?
      '<a href="https://openlibrary.org/borrow/
      ia/${bookInfo.ia}?ref=ol"
      class="btn btn-read" target="_blank">
      Read Book</a>' : ''}
  </div>
</div>';
});
$('.row.books').html(bookDiv)
})

```

В этом коде мы выбираем в переменную **authorsInfo** информацию об авторах. Дело в том, что у книги может быть один автор, а может быть несколько, поэтому параметр **authors** представляет собой массив, который нам тоже придется перебрать. Плюс данные в этом массиве содержат не только имя автора, но и ссылку на страницу с информацией о нем, поэтому мы формируем соответствующую разметку (рис. 29).

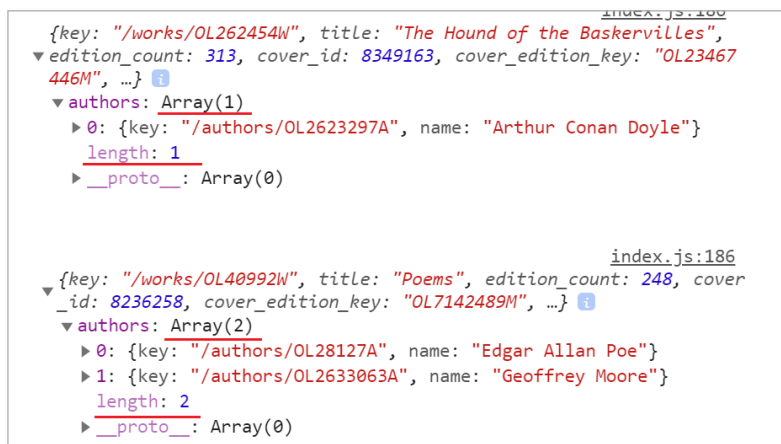


Рисунок 29

При формировании заголовка книги нам придется учесть, что помимо информации нам важен внешний вид блоков, поэтому слишком длинные названия мы будем укорачивать с помощью методов строк, чтобы заголовок уложился в 2 строки или в 60 символов.

```
<div class="book-title">
    ${bookInfo.title.length < 50 ? bookInfo.title :
      bookInfo.title.slice(0, 60) + '...'}
</div>
```

Вот, как это будет выглядеть:



Рисунок 30

Для каждого такого блока были написаны стили, поэтому разметка содержит ряд классов, описание которых вы найдете в css-файле. Один из блоков `<div class = "book-`

`hidden">` содержит информацию, которая появляется при наведении на блок с книгой. Там будут размещены кнопки ссылки на страницу с информацией о книге и на страницу, где эту книгу можно почитать онлайн. Однако, почитать можно не все книги, поэтому нам нужно проверить значение свойства `availability.is_readable` и в зависимости от этого выводить или же не выводить вторую кнопку-ссылку.

Works (Adventures of Sherlock Holmes / Case-Book of Sherlock Holmes)
Author: [Arthur Conan Doyle](#)

Read Book

```
{key: "/works/OL262554W", title: "Works (Adventures of Sherlock Holmes / Case-Book of Sherlock Holmes)", edition_count: 70, cover_id: 8351939, cover_edition_key: "OL26670244M", ...}
  authors: [{...}]
  availability:
    available_to_borrow: false
    available_to_browse: false
    available_to_waitlist: false
    collection: "digitallibraryindia,JaiGya..."
    identifier: "in.ernet.dli.2015.150489"
    is_browseable: false
    is_lendable: false
    is_printdisabled: false
    is_readable: true
    is_restricted: false
```

The Rainmaker
Author: [John Grisham](#)

More Info

```
{key: "/works/OL77021W", title: "The Rainmaker", edition_count: 63, cover_id: 9322929, cover_edition_key: "OL27112865M", ...}
  authors: [{...}]
  availability:
    error_message: "not found"
    identifier: "rainmakergrish00gris"
    is_browseable: false
    is_lendable: false
    is_printdisabled: false
    is_readable: false
    is_restricted: true
```

Рисунок 31

Последняя строка в этой функции выводит всю сформированную в переменной `bookDiv` разметку в подготовленный для этого пустой `<div class="row books"></div>`.

```
$('.row.books').html(bookDiv);
```

В коде присутствует еще один параметр `beforeSend` (перед отправкой) в виде функции, которая, как правило, используется для отображения предзагрузчика в виде `<div class="loader">` методом `show()`, пока браузер загружает файл, указанный в параметре `url`:

```
$.ajax({
  url: category_url,
  dataType: "json",
  beforeSend: function () {
    $('.loader').show();
  }
}).done(function (data) {
  $('.loader').hide();
  ...
})
```

В функции `done()`, которая обрабатывает загруженную информацию, мы предзагрузчик прячем методом `hide()`.

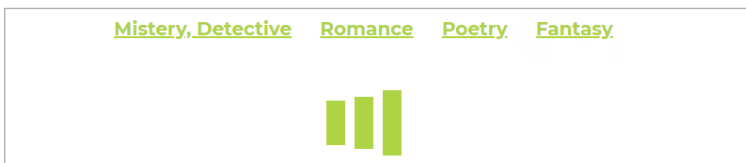


Рисунок 32

Для того чтобы посмотреть на то, что возвращает запрос на сервер, вы можете перейти на вкладку Network (Сеть)

в браузере и внизу кликнуть по названию загружаемого файла. Во вкладке **Preview** вы увидите ту же информацию, что и в параметре `data` функции `done()`.

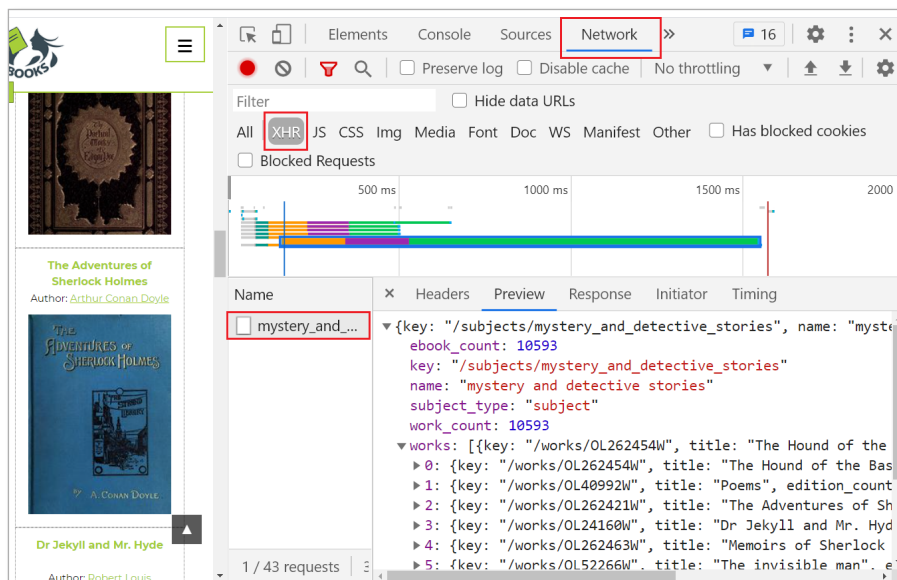


Рисунок 33

Обработка ошибок при AJAX-запросе

В том случае, если мы сделаем ошибку в адресе, ведущему к json-файлу, добавив, например, цифру 1 к адресу:

```
var category_url = "https://openlibrary.org/subjects/
mystery_and_detective_stories1.json";
```

API OpenLibrary все равно вернет нам данные в виде объекта, но в его свойстве `ebook_count` будет указан `0`. Эту ситуацию мы можем предусмотреть, добавив в функции

`done()`, обрабатывающей успешный запрос на сервер, такое условие:

```
if (data.ebook_count == 0) {
    $('.row.books').addClass('justify-content-center my-40')
    .html('<h3>No books found</h3>');
    return false;
}
```

Тогда на странице мы увидим сообщение о том, что книги не найдены, а во вкладке Network — ответ от сервера.

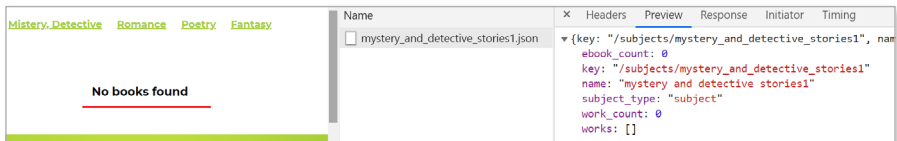


Рисунок 34

Однако ошибка может быть такой, что сервер вернет нам сообщение «Failed to load response data», т. е. «Не удалось загрузить данные ответа». Это может произойти, если в переменной `category_url` убрать расширение файла `.json`:

```
var category_url = "https://openlibrary.org/subjects/
    mystery_and_detective_stories";
```

В этом случае нам нужно будет написать код для функции `fail()`, которая как раз предназначена для обработки различных ошибок:

```
fail(function () {
    $('.loader').hide();
});
```

```
$('.row.books').addClass('justify-content-center
error my-3').html('<h3 class="text-center">
Loading of books failed...<br>An error has
occurred.</h3>')
});
```

Сообщение из этой функции выведется под нашим меню рубрик.

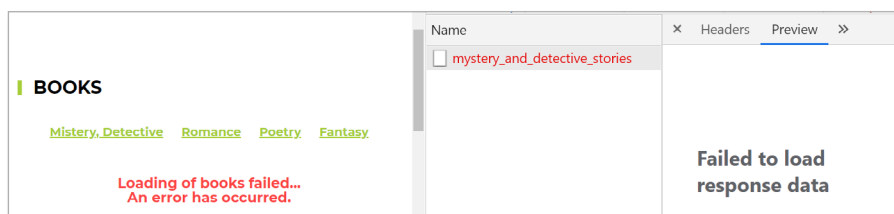


Рисунок 35

Домашнее задание

Ваше домашнее задание будет основано на файлах примера.

Задание 1

При уменьшении размера экрана изменяется вид меню. Вам необходимо будет сделать так, чтобы при клике на кнопку-гамбургер оно разворачивалось и сворачивалось с помощью одного из анимационных методов jQuery. Кроме того, внешний вид кнопки должен меняться: вместо 3-х полосок гамбургера при открытом меню должен появиться крестик и наоборот. Можно использовать для изменения кнопки html-спецсимволы `×` и `≡`.

Подсказка: можете использовать какой-нибудь класс, например `'open'` для открытого меню.



Рисунок 36

Задание 2

Обработайте клик на кнопке с текстом «That's great!» в разделе [#see-more](#), выведя рядом с ней текст 'You are 1025 reader now.', но получить цифру 1025 вы должны из атрибута `data-num="1024"`, увеличив его значение на 1. Используйте для этого метод [data\(\)](#), позволяющий работать с data-атрибутами, появившимися в HTML5, в jQuery, или метод [attr\(\)](#).

```
<button class="btn btn-read"
        id="readers-plus"
        data-num="1024">
    That's great!
</button>
<span id="num-of-readers"></span>
```

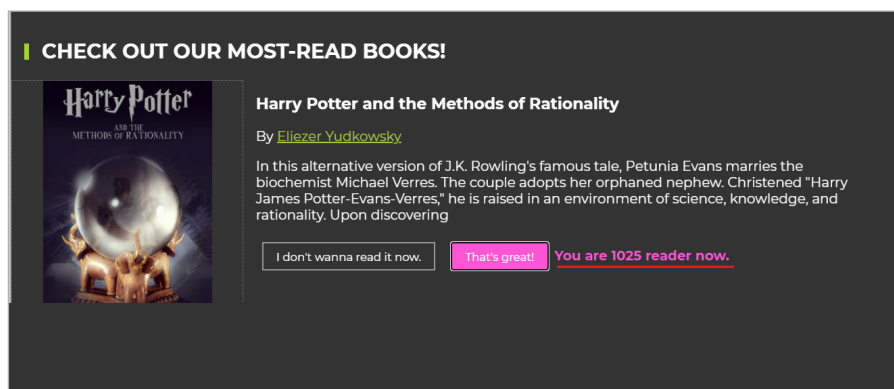


Рисунок 37

Через 1,5 секунды после появления этого сообщения перенаправьте пользователя с помощью `location.href` на страницу с адресом, указанным в ссылке с классом

`btn-read` внутри `<div class="book">` этого раздела. Получить ссылку нужно с помощью методов обхода дерева DOM, а не вставить ее в виде текста. Вы можете использовать метод `attr()` для того, чтобы добраться до значения атрибута `href` ссылки.

```
<a href="https://openlibrary.org/borrow/ia/hp
mor?ref=ol" class="btn btn-read" target="_blank">Read Book</a>
```

Задание 3

Преобразуйте код AJAX-загрузки книг одной рубрики в функцию и используйте ее для загрузки книг других рубрик при клике на ссылке в верхней части раздела **Books**. Используйте значение атрибута `href` каждой из этих ссылок, чтобы получить путь к JSON-файлу рубрики на OpenLibrary. Для этого вам нужно будет добавить к ним расширение `.json`.

```
<div class="categories text-center">
  <a href="https://openlibrary.org/subjects/mystery_and_detective_stories" class="c
  <a href="https://openlibrary.org/subjects/romance" class="cat-link">Romance</a>
  <a href="https://openlibrary.org/subjects/poetry" class="cat-link">Poetry</a>
  <a href="https://openlibrary.org/subjects/fantasy" class="cat-link">Fantasy</a>
</div>
```

Рисунок 38

Для активного элемента меню должен быть назначен класс `active`, чтобы его внешний вид поменялся (подчеркнуто на рисунках 39-40).

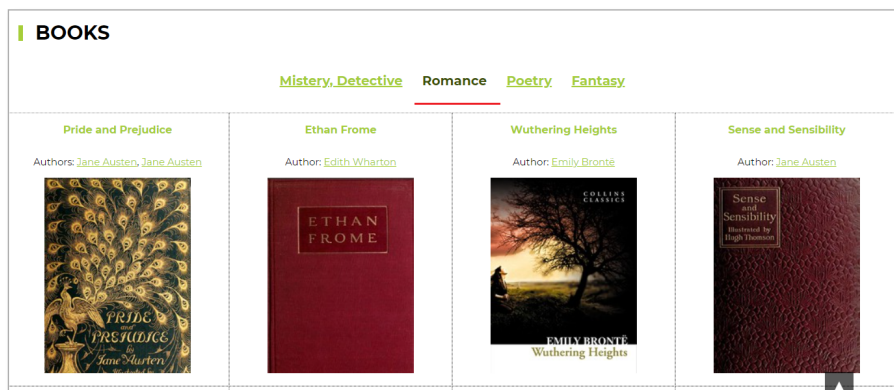


Рисунок 39

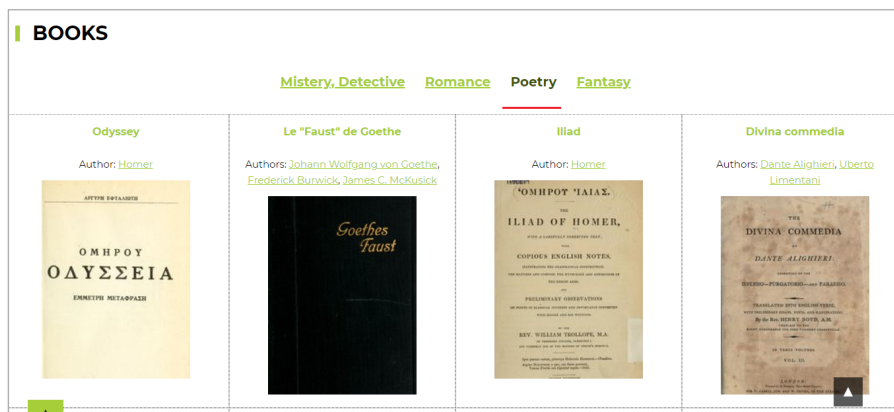


Рисунок 40

Приложение

Таблица 1. CSS-селекторы jQuery

Селектор	Описание	Пример
ПРОСТЫЕ СЕЛЕКТОРЫ		
<code>*</code> <code>_</code>	все элементы	<code>\$("*")</code>
<code>.className</code>	элементы с классом <code>className</code>	<code>\$(".bg-color")</code>
<code>#idName</code>	элемент (один!) с идентификатором <code>idName</code>	<code>\$("#test")</code>
<code>tagName</code>	элементы с заданным именем тега	<code>\$("h2")</code>
КОМБИНИРОВАННЫЕ СЕЛЕКТОРЫ		
<code>first, second, ...</code>	элементы удовлетворяющие любому из селекторов <code>first</code> , <code>second</code> , ...	<code>\$("h2, p")</code>
<code>outer inner</code>	элементы из <code>inner</code> , которые являются потомками (т.е. лежат внутри) элементов из <code>outer</code>	<code>\$(".test p")</code>
<code>parent > child</code>	элементы из <code>child</code> , которые являются прямыми потомками элементов из <code>parent</code>	<code>\$("#wrap > div")</code>
<code>prev + next</code>	элементы из <code>next</code> , которые следуют непосредственно за элементами из <code>prev</code>	<code>\$("label + input")</code>
<code>prev ~ next</code>	элементы из <code>next</code> , которые следуют за элементами из <code>prev</code>	<code>\$(".test ~ .block")</code>
СЕЛЕКТОРЫ АТРИБУТОВ		
<code>[name]</code>	элементы, содержащие атрибут <code>name</code>	<code>\$("[data-url]")</code> <code>\$("[data-dismiss]")</code>
<code>[name = value]</code>	элементы, у которых значение атрибута <code>name</code> совпадает с <code>value</code>	<code>\$("[name='radio-bg']")</code> <code>\$("[data-dismiss='true']")</code>

Таблица 1 (продолжение)

Селектор	Описание	Пример
<u>[name != value]</u>	элементы, у которых значение атрибута name не совпадает с value	<code>\$(".class!=test")</code>
<u>[name ^= value]</u>	элементы, у которых значение атрибута name начинается с value	<code>\$("a[href^='https://']")</code>
<u>[name \$= value]</u>	элементы, у которых значение атрибута name заканчивается на value	<code>\$(".class\$='-bg']")</code>
<u>[name*="value"]</u>	элементы, у которых значение атрибута name содержит value	
<u>[first][second][...]</u>	элементы, соответствующие всем заданным условиям на атрибуты одновременно	<code>\$("[name='bg'] [type='radio']")</code>
ФИЛЬТРЫ ДОЧЕРНИХ ЭЛЕМЕНТОВ		
<u>:first-child</u>	элементы, которые идут первыми в своих непосредственных предках	<code>\$(".block:first-child")</code>
<u>:last-child</u>	элементы, которые идут последними в своих непосредственных предках	<code>\$(".block:last-child")</code>
<u>:nth-child()</u>	элементы, которые расположены в своих непосредственных предках по определенным условиям	<code>\$(".block:nth-child(3n)");</code>
<u>:only-child</u>	элементы, которые являются единственными в своих непосредственных предках	<code>\$(".block img:only-child")</code>
<u>:only-of-type</u>	элементы, являющиеся единственными, потомками в своих родительских элементах	<code>\$("button:only-of-type")</code>
<u>:first-of-type</u>	те из выбранных элементов, которые первыми встречаются в своих родительских элементах	<code>\$("span:first-of-type")</code>

Таблица 1 (продолжение)

Селектор	Описание	Пример
<u>:last-of-type</u>	те из выбранных элементов, которые последними встречаются в своих родительских элементах	<code>\$('text:last-of-type')</code>
<u>:nth-last-of-type()</u>	те из выбранных элементов, которые являются определенными по счету относительно последнего элемента в своем элементе-родителе	<code>\$("ul li: nth-last-of-type(2)")</code>
<u>:nth-of-type()</u>	дочерние элементы определенного типа в определенной последовательности внутри родительского элемента	<code>\$(".column: nth-of-type(3n)")</code>

Таблица 2. jQuery-селекторы

Селектор	Описание	Пример
ПРОСТЫЕ ФИЛЬТРЫ		
<u>:first</u>	первый найденный элемент	<code>\$(".block:first")</code>
<u>:last</u>	последний найденный элемент	<code>\$(".block:last")</code>
<u>:eq(n)</u>	элемент идущий под заданным номером среди выбранных	<code>\$(".block:eq(3)")</code>
<u>:not(selector)</u>	все найденные элементы, кроме указанных в selector	<code>\$(":not(.color)")</code>
<u>:even</u>	элементы с четными номерами позиций, в наборе выбранных элементов	<code>\$(".block:even")</code>
<u>:odd</u>	элементы с нечетными номерами позиций, в наборе выбранных элементов	<code>\$(".block:odd")</code>
<u>:gt()</u>	элементы с индексом превышающим n	<code>\$(".block:gt(2)")</code>
<u>:lt()</u>	элементы с индексом меньшим, чем n	<code>\$(".block:lt(3)")</code>

Таблица 2 (продолжение)

Селектор	Описание	Пример
:header	элементы, являющиеся заголовками (с тегами h1, h2 и т. д.)	<code>\$(":header")</code>
:animated	элементы, которые в данный момент задействованы в анимации	<code>\$(".block:animated")</code>
<code>:hidden</code>	невидимые элементы страницы	<code>\$("form:hidden")</code>
<code>:visible</code>	видимые элементы страницы	<code>\$("form:visible")</code>
<code>:lang(language)</code>	элементы, в которых указаны языки содержимого	<code>\$("html:lang(ru)")</code>
<code>:root</code>	элемент, который является корневым в документе.	<code>\$(":root")</code>

Таблица 3. Селекторы форм

Селектор	Описание	Пример
:button	элементы с тегом <button> или типом button (<code><input type="button"></code>)	<code>\$(':button').addClass('shadow');</code>
:checkbox	элементы с атрибутом <code>type="checkbox"</code>	<code>\$(':checkbox').addClass('shadow');</code>
:checked	выбранные элементы (со статусом/атрибутом <code>checked</code>). Это могут быть элементы с атрибутом <code>type="checkbox"</code> или <code>type="radio"</code> .	<code>\$(':checked').addClass('shadow');</code>
:disabled	неактивные элементы формы (со статусом <code>disabled</code>)	<code>\$(':disabled').prop('disabled', false);</code>
:enabled	активные элементы формы (со статусом <code>enabled</code> или без атрибута <code>disabled</code>)	<code>\$(':enabled').addClass('bg-gray');</code>
:focus	элементы формы, находящиеся в фокусе	<code>\$(':focus').addClass('bg-red');</code>
:file	поля загрузки файлов (<code><input type="file"></code>)	<code>\$(':file').addClass('red-border');</code>
:image	элементы, являющиеся изображениями для отправки формы (<code><input type="image"></code>)	<code>\$(':image').prop('disabled', true);</code>

Таблица 3 (продолжение)

Селектор	Описание	Пример
:input	элементы, являющиеся элементами формы (с тегами input, textarea или button)	<code>\$('input').addClass('bg-green shadow')</code>
:password	поля для ввода пароля (<input type="password">)	<code>\$(':password').addClass('bg-red')</code>
:radio	элементы, являющиеся переключателями, т.е. имеющие type="radio"	<code>\$(':radio').addClass('shadow')</code>
:reset	кнопки для очистки формы (<input> или <button>)	<code>\$(':reset').addClass('shadow bg-gray');</code>
:selected	выбранные элементы (со статусом selected), как правило, элементы типа <option>	<code>\$(':selected').addClass('gray-border');</code>
:submit	кнопки для отправки формы (<input> или <button>)	<code>\$(':submit').addClass('shadow');</code>
:text	любые текстовые поля	<code>\$(':text').addClass('bg-blue');</code>

Примеры выбора элементов страницы в зависимости от используемых селекторов вы найдете на ресурсе [codepen.io](#) или в файле *examples/jquery-selectors.html*.

Для того чтобы увидеть, как действует селектор, вам необходимо будет сделать клик по любой радио-кнопке и обратить внимание на изменившиеся стили. В некоторых случаях над выбранным элементом будет появляться стрелка, указывающая на то, что изменилось.

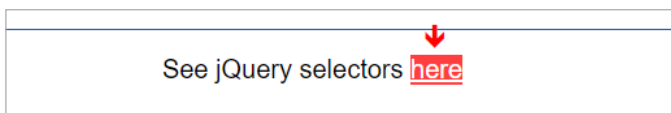


Рисунок 41

Чтобы понять, какие именно селекторы были использованы, придется заглянуть в js-код.

jQuery Selectors

CSS-selectors

☐ *	☐ #id	☐ .className	☐ tagName	☐ first, second, ...	☐ outer inner
☐ parent > child	☐ prev + next	☐ prev ~ next	☐ [name]	☒ [name = value]	☐ [name != value]
☐ [name ^= value]	☐ [name \$= value]	☐ [first][second][...]	☐ :first-child	☐ :nth-child	☐ :nth-child(2n+1)
☐ :only-child	☐ :only-of-type	☐ :first-of-type	☐ :nth-last-of-type(2)	☐ :nth-of-type(even)	

jQuery-selectors(filters)

☐ :first	☐ :last	☐ :not(selector)	☐ :even	☐ :odd	☐ :eq(n)
☐ :gt()	☐ :lt()	☐ :header	☐ :hidden	☐ :visible	☐ :root

jQuery Form elements selectors (filters)

☐ :button	☐ :checkbox	☐ :checked	☐ :disabled	☐ :input
☐ :radio	☐ :submit	☐ :reset	☐ :text	

Column 1

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Esse sequi eligendi iste namque.

Dolorem illo nostrum vel. Explicabo alias vel placeat lorem quaeat quis praesentium!

Debitis assumenda magni, ad mollitia repellat consequuntur eveniet molestiae nisi similique!

Column 2

Dolorem illo nostrum vel. Explicabo alias vel placeat lorem quaeat quis praesentium!

Ab rem at delectus, praesentium minus harum modi incididunt autem eaque quis velit accusamus?

Pariatur cupiditate doloremque commodi omnis ipsam modi incididunt, explicabo provident.

Debitis assumenda magni, ad mollitia repellat consequuntur eveniet molestiae nisi similique!

Column 3

Obcaecati voluptatibus amet esse at. Veniam accusamus, fugiat eligendi esse ullam architecto!

Pariatur cupiditate doloremque commodi omnis ipsam modi incididunt, explicabo provident.

Debitis assumenda magni, ad mollitia repellat consequuntur eveniet molestiae nisi similique!

Column 4

Beatae harum quasi dolore voluptates amet veritatis minima cumque quos nemo? Consequuntur.

Illo nostrum vel. Explicabo alias consequetur vel placeat minima cumque quos nemo? Consequuntur.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Commodi facere provident voluptas!

Column 5

Illo nostrum vel. Explicabo alias consequetur vel placeat minima cumque quos nemo? Consequuntur.

Error facere adipisci voluptates quia facilis, deleniti, aliquam a totam suscipit consequuntur?

Cum dolore perferendis iure possimus voluptatem similique mollitia repellat, ab omnis! Porro.

Contact Form

☒ Consultation ☐ Order ☐ Complaint

Your Name

Your Phone

Subject

Your Email

Your Message

See jQuery selectors [here](#)

Рисунок 42

Таблица 4. Методы управления стилями и классами jQuery

Метод	Описание	Пример
МЕТОДЫ УПРАВЛЕНИЯ СТИЛЯМИ JQUERY		
<code>\$('#селектор').css('property')</code>	получить значение указанного CSS-свойства	<code>\$('#block').css('display');</code>
<code>\$('#селектор').css('property', 'value');</code>	установить 1 значение	<code>\$('#block').css('color', 'red');</code>
<code>\$('#селектор').css({prop1: 'value1', 'prop2': 'value2', prop3: 'value3'});</code>	установить несколько значений. CSS-стили можно записывать в кавычках в нотации CSS или без кавычек в нотации JavaScript с помощью camelCase	<code>\$('.example').css({color: 'red', 'background-color': 'pink', fontSize: '18px'});</code>
МЕТОДЫ УПРАВЛЕНИЯ КЛАССАМИ JQUERY		
<code>\$('#селектор').addClass('имя_класса')</code>	Добавляет класс к списку классов элемента. Можно добавить несколько через пробел	<code>\$('#a').addClass('read-more'); \$('#btn').addClass('btn-outline btn-secondary');</code>
<code>\$('#селектор').removeClass('имя_класса')</code>	Удаляет класс из списка классов элемента. Можно удалить несколько через пробел	<code>\$('.nav-links.active').removeClass('active'); \$('.products').removeClass('col-md-4 col-sm-6');</code>
<code>\$('#селектор').toggleClass('имя_класса')</code>	Добавляет/удаляет класс(-ы) из списка классов элемента в зависимости от того, назначен ли соответствующий класс(-ы) элементу.	<code>\$('.nav-links').toggleClass('active'); \$('.products').toggleClass('col-md-4 col-sm-6');</code>
<code>\$('#селектор').hasClass('имя_класса')</code>	Проверяет, назначен ли указанный класс элементу. Если да, то возвращает true, в противном случае — false	<code>if(\$('.nav-links').hasClass('active')) { \$('.nav-links').parent().addClass('menu-active'); }</code>

Таблица 5. Traversing. Методы обхода DOM

Метод	Описание	Пример
<u>add()</u>	Добавление элементов в набор выбранных элементов	<code>\$('.column h3').add('p').addClass('red-border');</code>
<u>addBack()</u>	После поиска дочерних элементов, добавляет предыдущий набор элементов к текущему для назначения стилей или обработчиков событий	<code>\$('.column').find('p').addBack().addClass('red-border');</code>
<u>children()</u>	Возвращает все прямые дочерние элементы выбранного элемента	<code>\$('.column-selected').children().addClass('red-border');</code>
<u>closest()</u>	Возвращает первый предок выбранного элемента	<code>\$('.column').closest('.container').addClass('gray-border');</code>
<u>contents()</u>	Возвращает все прямые дочерние элементы выбранного элемента (включая узлы текста и комментария)	<code>\$('.column-selected').contents().addClass('shadow');</code>
<u>each()</u>	Выполняет функцию для каждого выбранного элемента	<code>\$('.column').each(function(ind, elem){ let columnText = \$(elem).find('p'); columnText.html(ind+' ' + columnText.text()); });</code>
<u>end()</u>	Завершает последнюю операцию фильтрации (поиска) вложенных элементов в текущей цепочке и возвращает набор элементов в предыдущее состояние	<code>\$('.column').find('.text').addClass('shadow').end().find('img').addClass('shadow');</code>
<u>eq()</u>	Возвращает элемент с указанным индексом в списке выбранных элементов. Индексы считаются с 0.	<code>\$('.column').eq(1).addClass('bg-blue');</code>

Таблица 5 (продолжение)

Метод	Описание	Пример
<u>filter()</u>	Возвращает набор совпадающих элементов, которые соответствуют указанному селектору или функции	<code>\$('.column img').filter('.snow').addClass('gray-border shadow');</code>
<u>find()</u>	Возвращает элементы-потомки выбранного элемента по указанному селектору	<code>\$('.column').find('.text').addClass('shadow');</code>
<u>first()</u>	Возвращает первый элемент среди выбранных	<code>\$('.column').first().addClass('bg-red');</code>
<u>has()</u>	Возвращает все элементы, которые имеют один или несколько элементов внутри них с указанным селектором	<code>\$('.column').has('img.snow').addClass('bg-blue');</code>
<u>is()</u>	Проверяет набор совпадающих элементов на объект Selector/элемент/jQuery и возвращает значение true, если хотя бы один из этих элементов соответствует заданным аргументам	<code>\$('.column').each(function(i, elem){ if(\$(this).is(':nth-child(2n)')) \$(this).addClass('bg-red'); });</code>
<u>last()</u>	Возвращает последний элемент из выбранных элементов	<code>\$('.column').last().addClass('bg-green');</code>
<u>map()</u>	Передаёт каждый элемент из набора с помощью функции, производя новый объект jQuery, содержащий возвращаемые значения	<code>\$('.container:eq(1)').prepend('<p class="text-center">Photo titles: '+ \$('.column img').map(function(ind, elem){ return \$(elem).attr('alt'); }).get().join(' ')+</p>');</code>
<u>next()</u>	Возвращает следующий родственный элемент выбранного элемента	<code>\$('.column-selected').next().addClass('bg-green');</code>
<u>nextAll()</u>	Возвращает все соседние элементы выбранного элемента	<code>\$('.column-selected').nextAll().addClass('bg-red');</code>

Таблица 5 (продолжение)

Метод	Описание	Пример
<code>nextUntil()</code>	Возвращает все соседние элементы одного уровня между двумя заданными аргументами	<code>\$('.column p').nextUntil('img').addClass('shadow');</code>
<code>not()</code>	Возвращает элементы, не совпадающие с определенными критериями	<code>\$('.column').not(':nth-child(2)').addClass('bg-green');</code>
<code>offsetParent()</code>	Возвращает первый позиционный родительский элемент	<code>\$('.column-selected').offsetParent().addClass('gray-border');</code>
<code>parent()</code>	Возвращает прямой родительский элемент для выбранного элемента	<code>\$('.column').parent().addClass('red-border');</code>
<code>parents()</code>	Возвращает все элементы-предки выбранного элемента	<code>\$('.column').parents('article').addClass('bg-red shadow');</code>
<code>parentsUntil()</code>	Возвращает все элементы-предки заданного элемента — не только непосредственного, но и прародителя, прапрародителя и так далее до корневого элемента (то есть до тега <code>html</code>).	<code>\$('.column').parentsUntil('container').addClass('bg-blue');</code>
<code>prev()</code>	Возвращает предыдущий родственный элемент выбранного элемента (-ов)	<code>\$('.column-selected').prev().addClass('bg-blue');</code>
<code>prevAll()</code>	Возвращает все предыдущие родственные элементы выбранного элемента	<code>\$('.column-selected').prevAll().addClass('bg-red shadow');</code>
<code>prevUntil()</code>	Возвращает все предыдущие элементы одного уровня между двумя заданными аргументами	<code>\$('.column p').prevUntil('img').addClass('shadow');</code>
<code>siblings()</code>	Возвращает все родственные элементы выбранного элемента	<code>\$('.column p:first-of-type').siblings().addClass('gray-border');</code>

Таблица 5 (продолжение)

Метод	Описание	Пример
slice()	Выбирает из набора элементов только те, чьи индексы указаны в диапазоне в скобках	<code>\$('column').slice(1,4).addClass('bg-red shadow');</code>
wrap()	Помещает выбранные элементы внутрь заданного элемента	<code>\$('column').wrap('<div class="bg-green shadow"></div>');</code>
wrapAll()	Оборачивает все выбранные элементы в общий элемент-обертку	<code>\$('column').wrapAll('<div class="bg-red shadow"></div>');</code>

В примере, который находится в папке *examples/jquery-traversing-methods.html* или на ресурсе [codepen.io](#), вы можете пощелкать по радио-переключателям, чтобы посмотреть работу всех перечисленных методов (рис. 43).

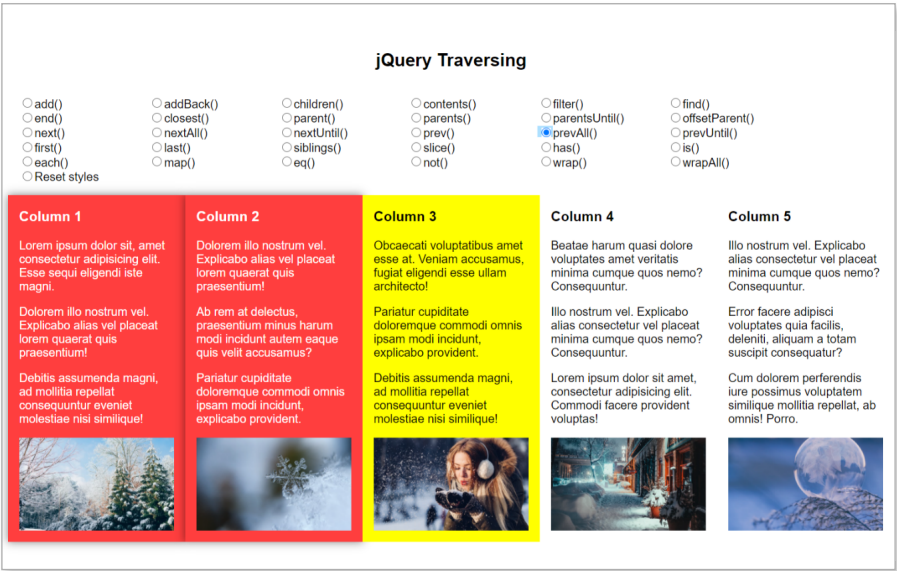


Рисунок 43

В коде добавляются различные классы со стилями, выделяющими какие-либо элементы в зависимости от выбранного метода.

По центру выделена колонка, относительно которой определяются элементы такими методами, как `prev()`, `prevAll()`, `next()` и др.

Код из этого файла использован в таблице с описанием методов в виде примеров.

Обратите внимание на то, как визуально изменяется разметка примера при использовании методов `wrap()` и `wrapAll()` (рис. 44).

Стоит заглянуть в Инспектор свойств браузера, чтобы увидеть, почему так происходит.



Рисунок 44

Метод `wrap()` обернул в `<div class="bg-green shadow">` `</div>` каждый из `<div class="column">` `</div>` (рис. 45).

Метод `wrapAll()` обернул в `<div class="bg-red shadow">` `</div>` все `<div class="column">` `</div>` (рис. 46).



Рисунок 45

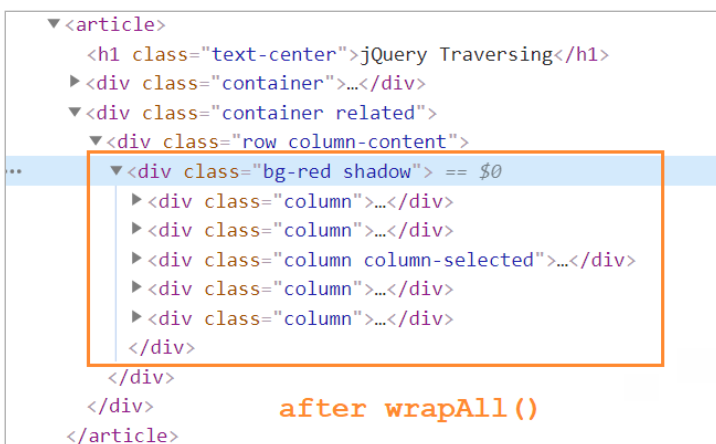


Рисунок 46